Efficient Heuristics for Classical Simulation of Quantum Circuits Using ZX-Calculus

Candidate no. TODO

Word count: TODO

Submitted in partial completion of the

Master of Science in Advanced Computer Science

☐ I hereby certify that this is entirely my own work unless otherwise stated.

Trinity 2024

Contents

List of Figures **iv**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Basic Notation . . . 2
1.3 ZX-Calculus . . . 5
1.3.1 Spiders . . . 6
1.3.2 ZX-Rules . . . 7
1.4 Graph Reduction . . . 9

2 Classical Simulation **13**
2.1 Introduction . . . 13
2.2 Strong Simulation . . . 14
2.3 Stabiliser Decomposition . . . 15
2.4 Cat States . . . 17
2.5 Graph Cuts . . . 20
2.6 Subgraph Complement . . . 22

3 Heuristics **24**
3.1 Introduction . . . 24
3.2 Greedy Algorithm . . . 26
3.3 Cut Heuristic . . . 27
3.3.1 CNOT Sandwich Heuristic . . . 29
3.3.2 Gadget $|\text{cat}_3\rangle$ Heuristic . . . 30
3.3.3 Lone Phase Heuristic . . . 34
3.3.4 Paired Heuristics . . . 34
3.3.5 Greedy Algorithm with Cut Heuristic . . . 36
3.3.6 Other Heuristics . . . 37
3.4 Complexity Analysis . . . 39

4 Results **41**
4.1 Benchmarking . . . 41
4.1.1 Random Clifford+T . . . 41
4.1.2 Random IQP . . . 43
4.1.3 Modified Hidden Shift . . . 46
4.1.4 Random Clifford+T with CCZ . . . 47
4.2 Experiments . . . 48
4.2.1 Random Clifford+T . . . 48
4.2.2 Random IQP . . . 48
4.2.3 Modified Hidden Shift . . . 50
4.2.4 Random Clifford+T with CCZ . . . 50

Appendices

References **63**

List of Figures

1.1 ZX-rules presented in [9] 8
1.2 Derived ZX-rules [12], where $\alpha, \beta, \gamma \in [0, 2\pi)$ and $j, k \in \{0, 1\}$. . . 10
1.3 A ZX-diagram in reduced gadget form [2] 12

2.1 Scalar involving Clifford+T diagram V 14

3.1 Example tree for a stabiliser decomposition 25
3.2 Diagram to showcase the heuristic (3.13) 32
3.3 Diagram where the heuristic (3.15) is not valid 32
3.4 Diagram where the heuristic (3.19) underestimates the effective heuristic value 34
3.5 A pair of vertices connected to 3-legged phase gadgets 35

4.1 An IQP circuit as a ZX-diagram, for $x_i, y_{i,j} \in \mathbb{Z}$, as defined in [6] . . 43
4.2 α values for all random Clifford+T circuits 48
4.3 Mean α values for different T-counts 49
4.4 Mean α values separated by number of qubits 49
4.5 Mean log number of terms for different T-counts, separated by number of qubits 50
4.6 Mean log time for different T-counts, separated by number of qubits 50
4.7 Improvement in α values for all random Clifford+T circuits 51
4.8 Improvement in log time for all random Clifford+T circuits 52
4.9 Number of completed simulations before timeout for $q = 7$ 52
4.10 Number of completed simulations before timeout for $q = 19$ 53
4.11 Number of completed simulations before timeout for $q = 50$ 53
4.12 Number of completed simulations before timeout for $q = 100$ 54
4.13 Mean β values separated by number of qubits 54
4.14 Improvement in β values for all random IQP circuits 55
4.15 Improvement in log time for all random IQP circuits 56
4.16 Mean β values separated by number of qubits, cut vs combined . . 56
4.17 Improvement in β values for all random IQP circuits, cut vs combined 57
4.18 Improvement in log time for all random IQP circuits, cut vs combined 58
4.19 α values for all modified hidden shift circuits 59
4.20 Mean α values of modified hidden shift circuits for different T-counts 59
4.21 Mean log time of modified hidden shift circuits for different T-counts 60
4.22 Improvement in log time for all modified hidden shift circuits 60
4.23 Number of completed simulations before timeout for $q = 7$ 61
4.24 Number of completed simulations before timeout for $q = 20$ 61
4.25 Number of completed simulations before timeout for $q = 50$ 61

1 Introduction

Contents

1.1 Motivation **1**
1.2 Basic Notation **2**
1.3 ZX-Calculus **5**
1.3.1 Spiders 6
1.3.2 ZX-Rules 7
1.4 Graph Reduction **9**

1.1 Motivation

Quantum computing, a field rooted in the principles of quantum mechanics, represents a revolutionary approach to computation. Unlike classical computing, which relies on binary operations performed on bits, quantum computing utilizes quantum bits, or qubits, capable of representing and processing information in ways that classical bits cannot. This unique capability of qubits to exist in superpositions and entangle with each other allows quantum computers to perform certain types of calculations exponentially faster than their classical counterparts. If a classical computer were limited by memory and runtime, a quantum computer could potentially bypass these scaling issues - dubbed the "quantum advantage".

If scalable quantum computers were to become a reality, they could solve some of the most challenging problems in computer science much more efficiently than classical computers. Classical computations play an essential role in assisting and accelerating the development of quantum computers, since classical simulation is a fundamental aspect of understanding and designing quantum hardware, often serving as the only means to validate these quantum systems [1]. Additionally, quantum algorithms can be implemented on classical simulators for testing and verification purposes while full-fledged quantum computers remain beyond reach (for now). These simulators emulate the behavior of quantum computers, allowing researchers to simulate processes as if running on actual quantum devices.

This thesis aims to present the speedup of classical simulation of quantum circuits in an almost entirely graphical approach. From the introduction of the

qubits, to the representation of quantum circuits and linear maps, the underlying linear algebra that governs the operations of quantum computing is represented in a graphical manner. Using ZX-Calculus, a graphical calculus detailed in Section 1.3, quantum circuits can be represented as diagrams, allowing for different discoveries to be more easily made when analysing the structure of said diagrams.

The thesis is organised as follows. First, from this chapter, we introduce the basic notation and concepts of quantum computing, with a focus on the ZX-Calculus. Next, Chapter 2 outlines the methods used for classical simulation via stabiliser decomposition using ZX-Calculus as covered in [2–4]. Chapter 3 formalises the definition of using heuristics to improve classical simulation, and connects these heuristics to the existing methods in the literature [4–6]. The classes of circuits benchmarked are introduced in Chapter 4, as well as the numerical results of using heuristics covered in Chapter 3.

TODO: details of last chapters

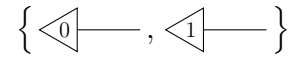
1.2 Basic Notation

This section, and the one after it, introduce the basic notation and concepts of quantum computing, along with the ZX-Calculus, and is a subset of the concepts covered by Coecke and Kissinger in Picturing Quantum Processes [7].

In classical computing, a bit is the fundamental unit of information, and is denoted by a boolean value $b \in \{0, 1\}$. In quantum computing, the fundamental unit of information is the qubit, which we denote as $|b\rangle$, where $b \in \{0, 1\}$. Diagramatically, they are represented as follows:

We consider these the computational or Z-basis states.

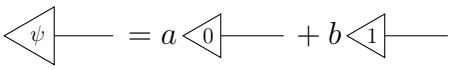
Definition 1.2.1 (Z-basis)**.** The *Z-basis* or computational basis is defined as



▲

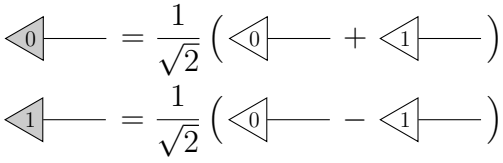
So far, the expressive power is still similar to classical computing. However, the power of quantum computing comes from the ability to exist in superpositions of these states. Concretely, the qubit can be represented in a linear combination of the basis states.

Definition 1.2.2 (Pure state)**.** A *pure state* $|\psi\rangle$ is a state



where $a, b \in \mathbb{C}$ and $|a|^2 + |b|^2 = 1$. ▲

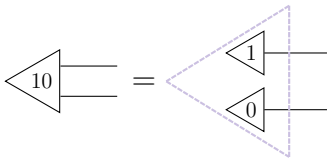
Then, the '+' or X-basis states can be defined.



(1.1)

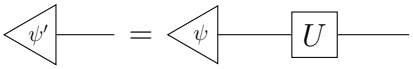
Along with the Z-basis states, these states form the building blocks of understanding quantum computing and the use of ZX-calculus for diagrammatic reasoning.

Intuitively, using the diagrammatic representation, we can consider combining diagrams in parallel. In the literature, this is the same as taking a *tensor product* $\otimes$. For example, we can have a 2-qubit state.



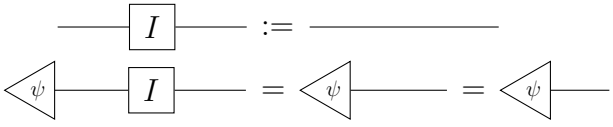
(1.2)

States can be transformed by quantum gates simply by *composing* them to obtain a new state. Given that composing the quantum gate U to a state $|\psi\rangle$ gives a state $|\psi'\rangle$, we can just connect the wires.



(1.3)

Intuitively, this means that the wire by itself should just be the identity gate, since composing it with any state should not change that state.

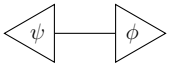


(1.4)

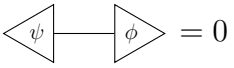
The dual of states are the effects. Here, we have the Z-basis effects.

For a state $|\psi\rangle$ and an effect $\langle\phi|$, we can obtain a scalar inner product. This is essentially the *bra-ket* notation in diagrammatic form by just connecting the wires, which comes with a few other definitions.

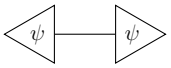
Definition 1.2.3 (Inner product [7])**.** The *inner product* $\langle\phi|\psi\rangle$ of two states $|\psi\rangle$ and $|\phi\rangle$ is defined as



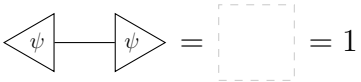
They are *orthogonal* if



The *squared-norm* of a state $|\psi\rangle$ is



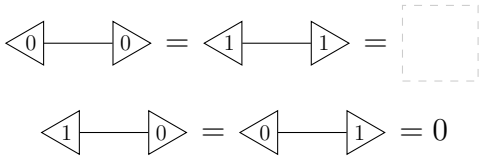
$|\psi\rangle$ is *normalised* if



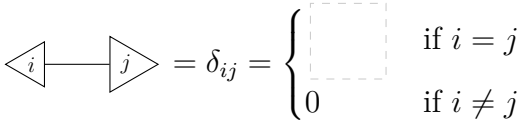
where the empty diagram represents the scalar 1. ▲

We can then define the inner product of the Z-basis states.

Definition 1.2.4. The inner product of the Z-basis states is



Succinctly, for $i, j \in \{0, 1\}$,

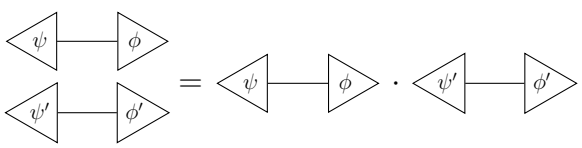


where δ_{ij} is the Kronecker delta. ▲

In fact, the succinct definition can be used to determine if a basis is an *orthonormal basis* (ONB). The X-basis is also an ONB.

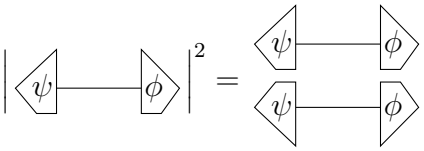
We are almost at the stage where we can obtain probabilities from the diagrams. The scalar inner product is not really a probability, that is, a number between 0 and 1, but instead, a number $z \in \mathbb{C}$.

First, we note that putting multiple scalars in a single diagram is just the same as multiplying them together.



(1.5)

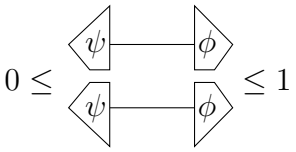
We also know that for a complex number and its conjugate, $z, \bar{z} \in \mathbb{C}, |z|^2 = z\bar{z}$ Translating that to the diagrammatic representation, we can obtain the following.



(1.6)

Note that assymetry is introduced to denote the conjugation. Remember that a complex number $z = a + bi \in \mathbb{C}$ has a conjugate $\bar{z} = a - bi \in \mathbb{C}$, so this corresponds to a vertical reflection in the diagram above.

Using the *Born rule*, if we make sure that $|\psi\rangle$ and $|\phi\rangle$ are normalised, we can obtain the following.



(1.7)

This can be interpreted as our desired probaility - the probability of measuring $|\psi\rangle$ in the state $|\phi\rangle$.

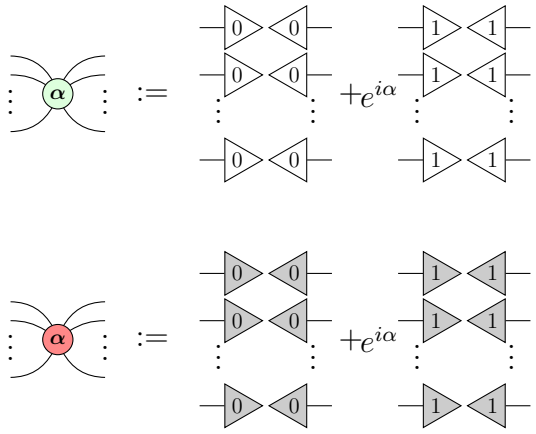
We now have all we need to introduce the ZX-calculus notation.

1.3 ZX-Calculus

So far, we have introduced the representation of quantum states, effects, and inner products in a diagrammatic form. However, there still isn't much we can do without a way to manipulate them. In the previous section, we mentioned that we can apply quantum gates to states to obtain new states. Here, we will introduce the ZX-Calculus and show the highly intuitive representation of some quantum gates such as the Hadamard and CNOT gates, based on the building blocks from the previous section. The ZX-Calculus is a graphical calculus for quantum computations, developed by Coecke and Duncan in 2008 [8]. All operations are expressed as *spiders*, connected by *edges* (or *wires*).

1.3.1 Spiders

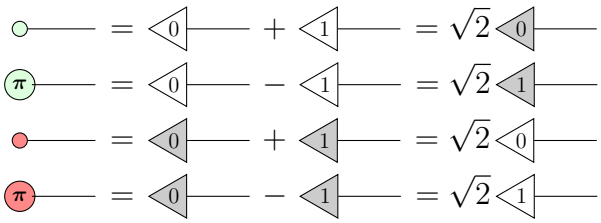
Definition 1.3.1 (Spiders)**.**



▲

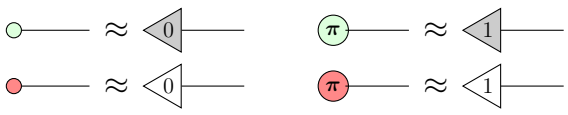
Here, we have green and red spiders, also known as Z and X spiders, made of the Z and X basis states respectively. Wires coming in from the left are *inputs*, and wires going out to the right are *outputs*. The α inside a spider node is also called the *phase* of the spider. For example, with $\alpha = 0, e^{i\alpha} =$. It is convenient to denote spiders with $\alpha = 0$ as having no angle written inside the spider.

Immediately, we can see that the Z and X basis states can be formed from the definition of the spiders, up to the multiplicative constant $\sqrt{2}$. Note that $e^{i\pi} = -1$.



(1.8)

It turns out that in reasoning with spiders, we can ignore the multiplicative constants, since much of the manipulation of the diagrams is based on the structure of the diagrams themselves. The scalars are easy to determine using the definitions we have seen so far. By using $\approx$ to denote equality up to a non-zero factor, we have

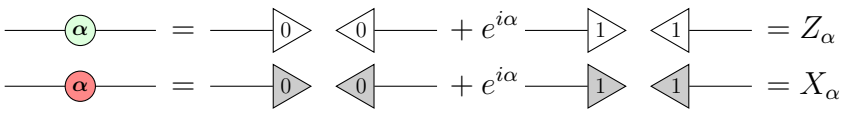


(1.9)

Scalars can also be represented using just spiders.

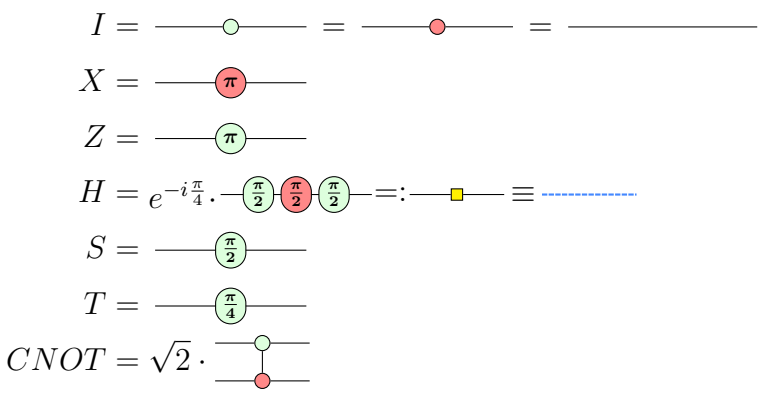
(1.10)

We can also see the single-qubit gates in the ZX-Calculus.



(1.11)

We can then define many of the common quantum gates in this way.

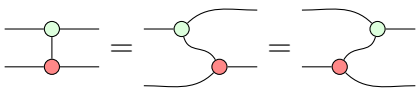


(1.12)

Here, we define a Pauli gate as any gate $P \in \{I, X, Y, Z\}$. There are 2 gates of special note here. The Hadamard gate H, denoted by the small yellow box, or as a blue dashed line, is a single-qubit gate that interchanges Z and X bases. We will see later in the ZX-rules that this is vital for the colour change (***cc***) rule, as well as why it is useful to denote it as the blue dashed line.

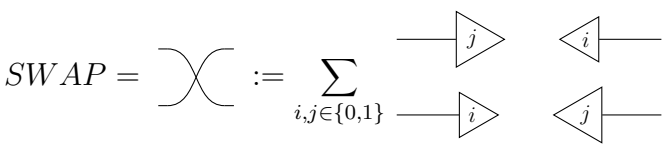
The CNOT gate is a two-qubit gate that acts on the target qubit if the control qubit is in the state $|1\rangle \approx$ 

. The vertical line is used simply because it does not matter which direction the line is going, where the following equality of the second and third forms can be verified.



(1.13)

Last but not least, we also introduce the SWAP gate, which as the name suggests, swaps two qubits.



(1.14)

We can now introduce the ZX-rules in the next section.

1.3.2 ZX-Rules

The ZX-Calculus is based on a set of rules that allow us to manipulate the diagrams. For $\alpha, \beta \in [0, 2\pi)$ and $a \in \{0, 1\}$, the rules are as follows.

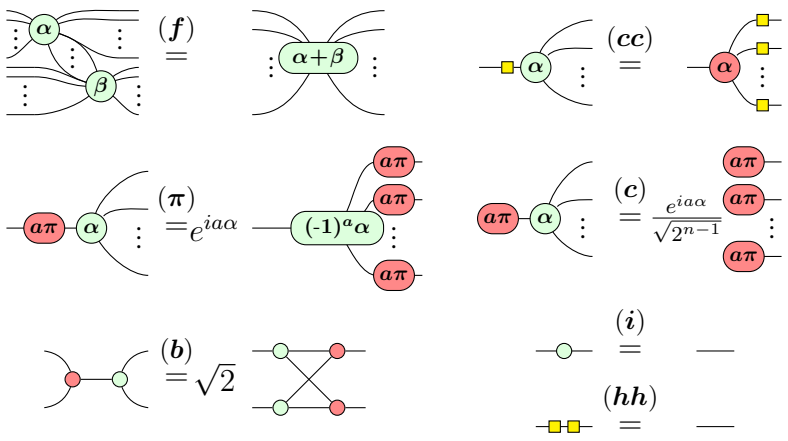


Figure 1.1: ZX-rules presented in [9]

The names of the rules are spider (***f***)usion, colour change (***cc***) rule, (***π***)-commutation, (***c***)opy rule, (***b***)ialgebra, (***i***)dentity rule, and (***hh***)adamard cancel rule. For (***c***), n refers to the number of outputs. By duality, the rules also hold if the colours are swapped.

The power of ZX-diagrams comes from the fact that two diagrams are equivalent as long as the connectivity of the diagram is the same - that is, as long as order of inputs and outputs of the whole diagram is preserved. This does not depend on the position of the spiders or the direction of wires between them. This neat feature of ZX-diagrams is called "Only Connectivity Matters" (OCM).

We can then obtain caps and cups using 2-legged spiders together with (***i***).

(1.15)

Caps and cups obey the *yanking equations*.

(1.16)

Before we move on to reasoning about the diagrams in a graph-theoretic manner, we first introduce the concept of a *Clifford+T diagram*.

Definition 1.3.2 (Clifford diagram)**.** A *Clifford diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{2}$. ▲

Definition 1.3.3 (Clifford+T diagram)**.** A *Clifford+T diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{4}$. ▲

It is clear from the definitions that a Clifford diagram is a subset of a Clifford+T diagram. The reason we introduce these concepts is that in trying to obtain our main goal of the thesis, it is sufficient to reason about Clifford+T diagrams, as a result of the following theorem.

Theorem 1.3.4 (Approximate Universality [10])**.** Clifford+T diagrams are *approximately universal* for quantum computing - they can approximate any quantum circuit.

The proof of the above theorem involves the *Solovay-Kitaev theorem*, combining with the fact that any single-qubit phase gate can be approximated by a sequence of Hadamard and T gates, with details found in [10].

1.4 Graph Reduction

This section connects the ZX-diagrams to graphs, in a way which will continue to be used for the rest of the thesis. The content in this section will be covered by Kissinger and van de Wetering in the soon-to-be-published Picturing Quantum Software [11]. Most of the concepts were also covered in [12, 13]

Because of OCM, ZX-diagrams up until now have very similar structure to graphs, where the spiders are the vertices and the wires are the edges. The only difference may be that the spiders are coloured, and that the Hadamard boxes are included. In this section, we solidify the connection between ZX-diagrams and graphs, and introduce an algorithm that will simplify ZX-diagrams, defining what it means for a graph to be simplified. Ultimately, simplifying ZX-diagrams should allow us to reduce the number of spiders in the diagram, which directly translates to a reduction in the number of gates in the quantum circuit. In turn, as we will see in the next chapter, this will lead to a speedup in the simulation of quantum circuits.

First, we define what it means for ZX-diagram to be *graph-like*.

Definition 1.4.1 (Graph-like [11])**.** A ZX-diagram is *graph-like* when

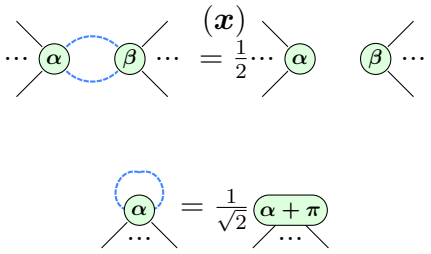
- Every spider is a Z-spider.
- Spiders are only connected via Hadamard edges.
- There is at most only one edge between each pair of spiders.
- There are no self-loops.
- Every input and output wire is connected to a Z-spider.

▲

Just from this definition, Hadamard edges are in fact the only edges in the graph-like ZX-diagram, leading to the convenience of having them as the blue dashed lines introduced earlier in (1.12). The following proposition can then be proven, using rewrite rules introduced previously in ZX-Rules.

Proposition 1.4.2 ([11])**.** Every ZX-diagram can be converted to a graph-like ZX-diagram in polynomial time.

The proof [11, 12] makes use of the fact that Hadamard edges in parallel cancel, and that we can always remove a Hadamard self-loop. Parallel edges cancelling is an application of the *Hopf rule* (***x***), derivable from the ZX-rules [1].



(1.17)

We can simplify the graph-like ZX-diagram further. Now, the following two rewrite rules, derivable from the original ZX-rules, are introduced.

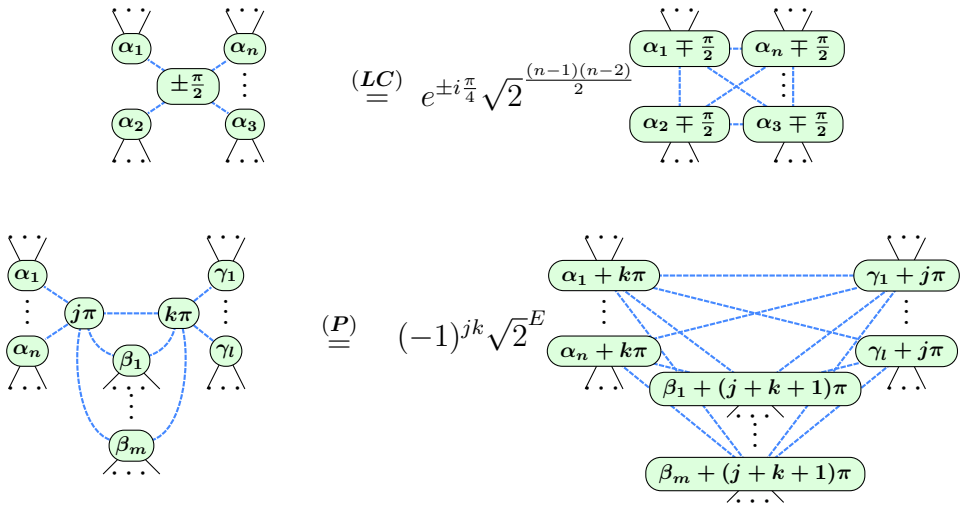


Figure 1.2: Derived ZX-rules [12], where $\alpha, \beta, \gamma \in [0, 2\pi)$ and $j, k \in \{0, 1\}$.

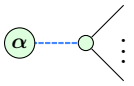
The rules are *local complementation* (***LC***) and *pivoting* (***P***) respectively. Here, $E = (n-1)m + (l-1)m + (n-1)(l-1)$.

[1] In fact, (***b***) $\implies$ (***x***)

Repeatedly simplifying and reducing diagrams in this manner will end up giving us patterns in the diagrams. The following definitions help us identify these patterns. First, we define interior and boundary spiders, and then phase gadgets.

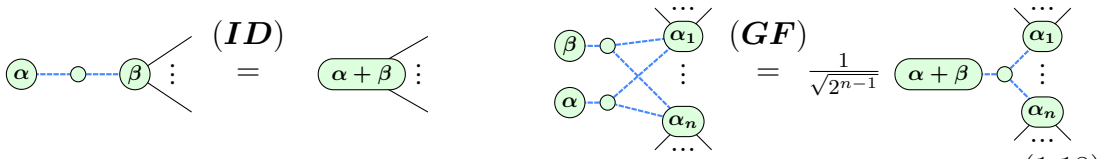
Definition 1.4.3 ([13])**.** A *boundary spider* is a spider connected to an input or output. All other spiders are *internal spiders*. ▲

Definition 1.4.4 (Phase gadget [13])**.** A *phase gadget* is an arity-1 spider with angle α, connected via a Hadamard edge to a spider with no angle.



▲

The usefulness of phase gadgets is that they can be used to simplify the diagram further. Here, two more derived rules involve phase gadgets.



(1.18)

These help us define the reduced gadget form.

Definition 1.4.5 (Reduced gadget form [13])**.** A graph-like ZX-diagram is in *reduced gadget form* when

- Every internal spider is a non-Clifford spider or part of a non-Clifford phase-gadget.
- Every phase-gadget has more than one target.
- No two phase-gadgets have the same set of targets.

▲

It turns out that there is an algorithm detailed below that is bounded in $O(n^3)$ where n is the number of spiders, to convert any graph-like ZX-diagram to reduced gadget form [2]. Thus, much of the work in this thesis will focus on the reduced gadget form, as it tends to bring us closer to the goal of reducing overall T-count. However, there are exceptions to this, as we will see in the next chapter.

Algorithm 1.4.6 (ZX-simplify [2])**.** Starting with a graph-like ZX-diagram, do the following:

1. Apply (***LC***) until all proper Clifford spiders are removed.
2. Apply (***P***) (and variations) until all interior $k\pi$-spiders are removed or transformed into phase-gadgets.
3. Remove all Clifford phase-gadgets using (***LC***) and (***P***).
4. Apply (***ID***) and (***GF***) wherever possible. If any matches are found, go back to step 1. Otherwise, the diagram is in reduced gadget form.

Example 1.4.7. The following diagram from [2] is in reduced gadget form. ▲

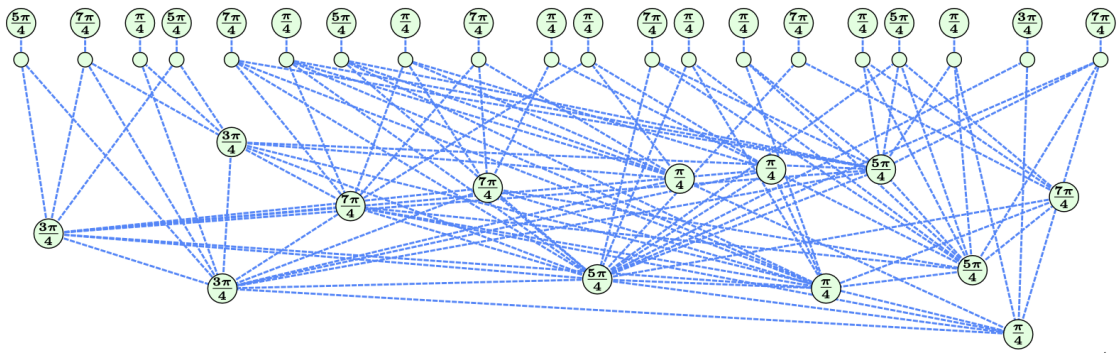


Figure 1.3: A ZX-diagram in reduced gadget form [2]

2
Classical Simulation

Contents

2.1 Introduction 13
2.2 Strong Simulation 14
2.3 Stabiliser Decomposition 15
2.4 Cat States 17
2.5 Graph Cuts 20
2.6 Subgraph Complement 22

This chapter introduced the concepts required for classical simulation, using stabiliser decompositions, magic states, cat states, and graph cuts. There is no novel work in this chapter, with all results being adapted from existing literature [2–4, 14, 15].

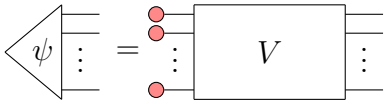
2.1 Introduction

As mentioned in the previous chapter, classical simulation of quantum circuits provides an essential tool for verification and validation, despite its inherent limitations. This chapter delves into the methods and advancements in classical simulation via stabiliser decomposition, discussing the role of magic states, cat states, and graph cuts, in extending the capabilities of these simulations. Classical simulation methods that store a complete description of an n-qubit quantum state as a complex vector of size 2^n are effective only for a small number of qubits, typically around 30. For example, Shor's factoring algorithm has been simulated with 31 qubits and approximately half a million gates [16]. However, for certain classes of quantum circuits, such as those that can be represented in a Clifford diagram, classical simulation can be significantly more efficient [17]. This is known as the *Gottesman-Knill theorem*.

Theorem 2.1.1 (Gottesman-Knill Theorem)**.** A Clifford computation can be efficiently classically simulated.

In other words, a computation involving only *Clifford diagrams* can be efficiently classically simulated. This is extremely useful when considering that ZX-Calculus simplifies the process of identifying and manipulating *stabiliser states*.

Definition 2.1.2 (Stabiliser state [15, 17])**.** An n-qubit state $|\psi\rangle$ is a *stabiliser state* if it has the form

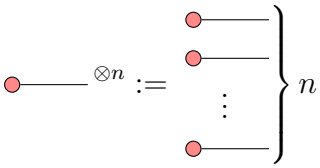


where V is a Clifford diagram. ▲

It is clear that if a computation only involves stabiliser states, then it can be efficiently classically simulated. Stabiliser circuits are particularly notable for their applications in quantum error correction [17]. In the next few sections, we will see more clearly how these stabiliser states are used in classical simulation. Particularly, we also need to consider Clifford+T diagrams, and how we can still (inefficiently) simulate them using stabiliser states.

2.2 Strong Simulation

When simulating quantum circuits, we assume that the input state to our n-qubit circuits is of the form



(2.1)

Given that we have a n-qubit circuit with the input state from (2.1), we would then want to obtain the probability of measuring a certain output state. For instance, we can plug in the outputs $\langle\vec{x}|$ for $\vec{x} = x_1, x_2, ..., x_n$ as spiders as in Figure 2.1.

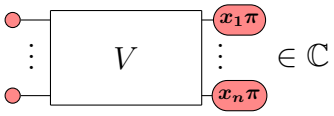
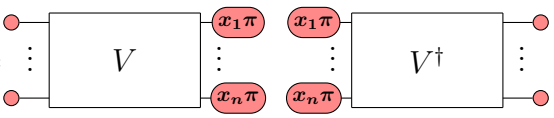


Figure 2.1: Scalar involving Clifford+T diagram V

Then using the Born rule, we can obtain the probability of measuring the output state [11].

$$\Pr(x_1, x_2, ..., x_n) = \frac{1}{\sqrt{2^{4n}}}$$



(2.2)

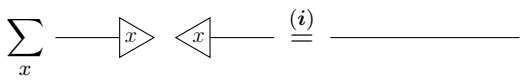
$V^\dagger$ is used to denote the adjoint of V, which simply means the inputs and outputs are swapped, and the spider phases are negated. The scalar factors of $1/\sqrt{2}$

comes from those seen in (1.8). If V is a Clifford diagram, then following from the Gottesman-Knill Theorem, the scalars can be computed efficiently. Obtaining the probabilities this way is known as *strong simulation*.

Marginal probabilities can also be obtained similarly. Recall that using the law of total probability, we have for $k < n$,

$$\Pr(x_1, x_2, ..., x_k) = \sum_{x_{k+1}, x_{k+2}, ..., x_n} \Pr(x_1, x_2, ..., x_k, x_{k+1}, ..., x_n) \tag{2.3}$$

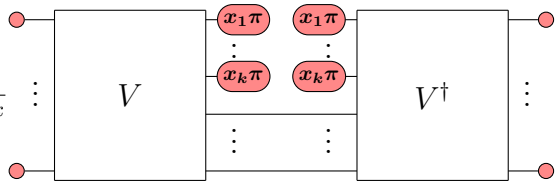
Then, applying the (2.3) to (2.2), and using the (***i***)dentity rule, where



(2.4)

we can obtain the marginal probabilities.

$$\Pr(x_1, x_2, ..., x_k) = \frac{1}{2^{n+k}}$$

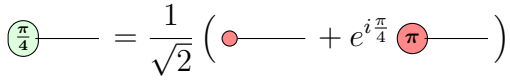


(2.5)

A natural question that arises is whether the same can be done if V is a Clifford+T diagram instead. The following sections will delve into the methods used to simulate such circuits.

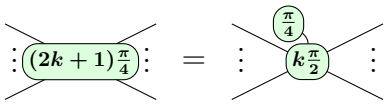
2.3 Stabiliser Decomposition

Definition 2.3.1 (Magic state [15])**.** The *magic state* is



▲

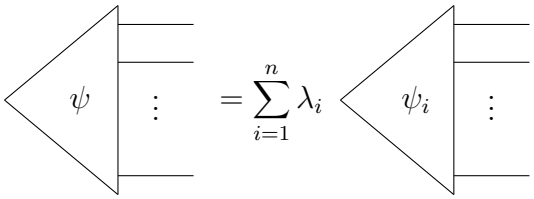
The naïve approach to make use of this magic state decomposition is to then apply it to every single T-like spider in the Clifford+T diagram, where we define a T-like spider as a spider with a phase of $(2k+1)\frac{\pi}{4}$ for some $k \in \mathbb{Z}$. By un(***f***)using the magic state from these spiders as below, we can apply the Magic state [15] decomposition to obtain a sum of stabiliser states.



(2.6)

This sum can be called the *stabiliser decomposition* of the circuit.

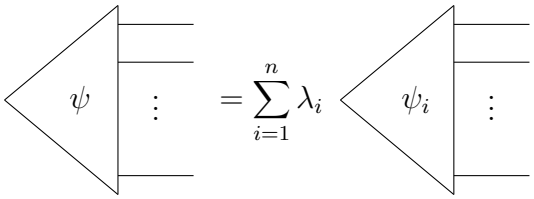
Definition 2.3.2 (Stabiliser Decomposition [15])**.** The *stabiliser decomposition* of a state $|\psi\rangle$ is a sum of stabiliser states



where $\lambda_i \in \mathbb{C}$ and $|\psi_i\rangle$ are stabiliser states. ▲

More generally, a decomposition need not consist only of stabiliser states.

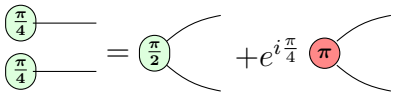
Definition 2.3.3 (Decomposition)**.** The *decomposition* of a state $|\psi\rangle$ is a sum of states



where $\lambda_i \in \mathbb{C}$ and $|\psi_i\rangle$ are states. For any decomposition d, define its length to be $|d| = n$. ▲

Using the Magic state [15] decomposition, we can see that the value of $n = 2^t$ if we apply it to the t T-like spiders in the circuit. We can call t the *T-count* of the circuit.

However, there is an even better decomposition



(2.7)

This allows us to decompose the circuit into a sum of stabiliser states with $n = 2^{\frac{t}{2}}$. Different methods of decomposing the circuit can lead to different values of n. Since n is believed to be exponential in t (or else we have $P = NP$) [18], we can take it that $n = O(2^{\alpha t})$ for some $0 < \alpha < 1$. We can then compare the efficiency of different methods by using this metric.

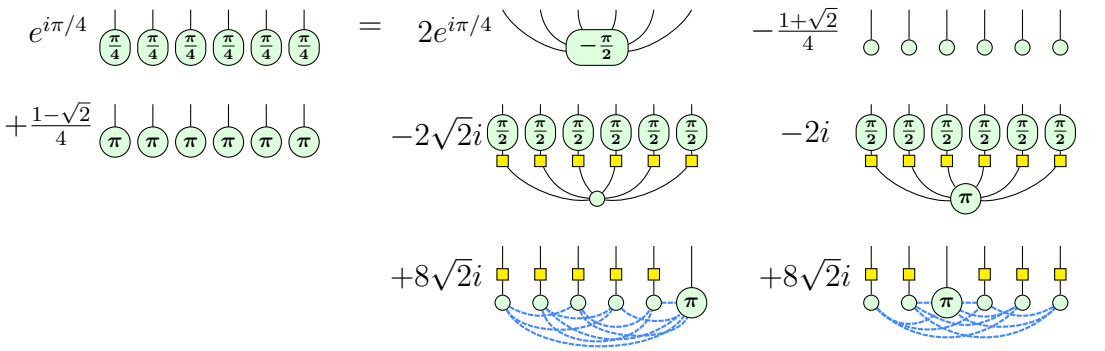
Definition 2.3.4 ((In)efficiency [2, 3, 15, 19])**.** The *(in)efficiency* of a decomposition D that reduces the T-count by r using p states is

$$\alpha(D) := \frac{\log_2(p)}{r} \tag{2.8}$$

where a lower value of α [1] indicates a more efficient decomposition. ▲

[1] We omit D when the context is clear

For example, the Bravyi-Smith-Smolin (BSS) decomposition [15] makes use of the following decomposition which has $\alpha \approx 0.468$.



(2.9)

Using this metric, we can even compare the efficiency of different algorithms used to decompose the circuit. It also allows us to obtain the efficiency of different decompositions which do not use magic states at all. As we will see in the next few chapters, this ties in well when we consider the higher efficiency of *cat state* decomposition.

A closely related metric we can use is the *stabiliser rank*.

Definition 2.3.5. Let D be the set of all stabiliser decompositions of a state $|\psi\rangle$. The *stabiliser rank* $\chi(|\psi\rangle)$ of a state $|\psi\rangle$ is

$$\chi(|\psi\rangle) := \min\{n \mid \exists(d_1, d_2, ..., d_n) \in D \text{ s.t. } |\psi\rangle = \sum_{i=1}^{n} d_i\}$$

▲

We can see that the relation between the stabiliser rank and its efficiency is

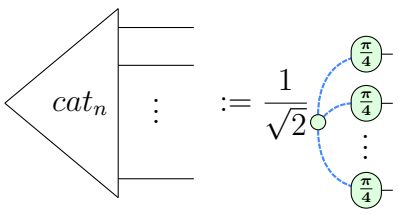
$$\chi(|\psi\rangle) = 2^{\alpha_\chi(|\psi\rangle)t} \tag{2.10}$$

with the T-count t of the circuit.

The goal for any stabiliser decomposition is to obtain an efficiency as close to α_χ as possible.

2.4 Cat States

Definition 2.4.1 (Cat state [14])**.** A *cat state* is defined as

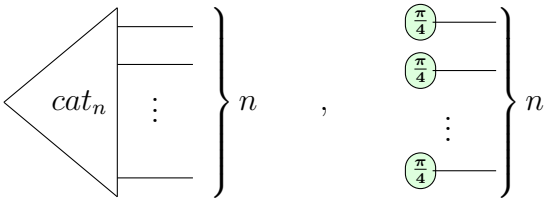


(2.11)

▲

Cat states are used because their decompositions have low value of α, as well as the following proposition, which bounds the α of the corresponding magic state decomposition.

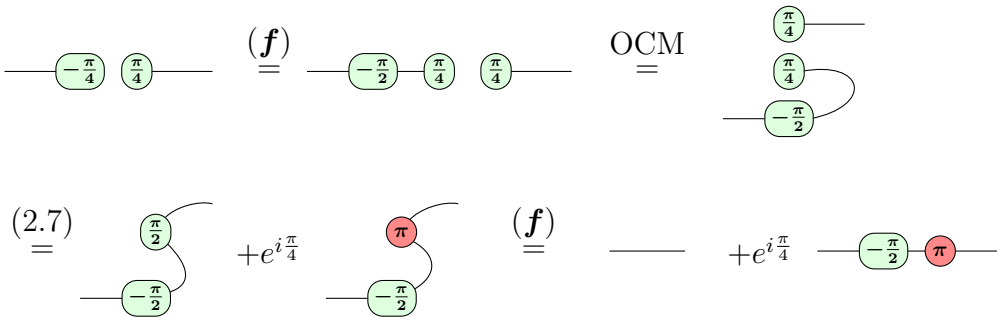
Proposition 2.4.2. Suppose α_1, α_2 are the efficiency metrics of the decompositions of



respectively. Then

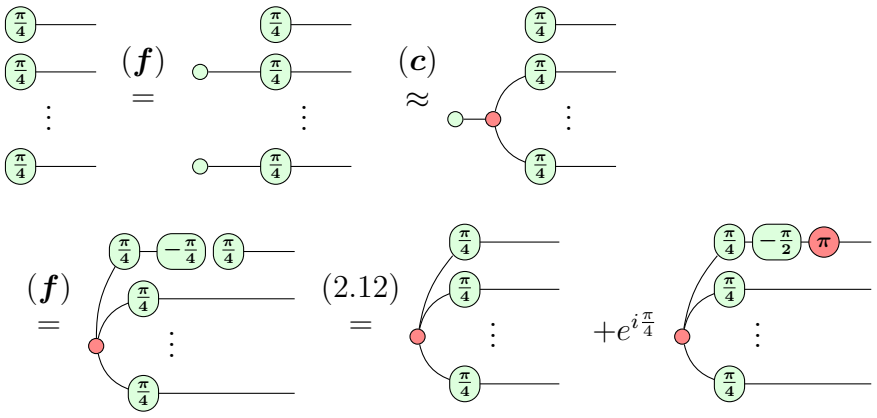
$$\alpha_1 \leq r \implies \alpha_2 \leq r + \frac{1}{n}$$

Proof. The proof is adapted from [14]. Suppose $\alpha_1 \leq r$. First, we show the following equality.



(2.12)

Then we can show the following.



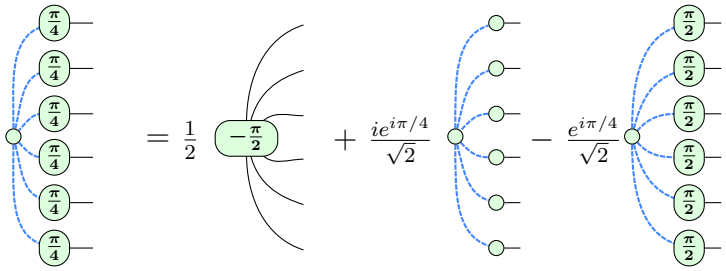
(2.13)

There are 2 copies of $|\text{cat}_n\rangle$ which would each give at most $2^{n\alpha_1}$ stabiliser states, for a total of at most $2^{n\alpha_1+1}$ stabiliser states. Thus, we have

$$\alpha_2 \leq \frac{n\alpha_1 + 1}{n} \leq r + \frac{1}{n}$$

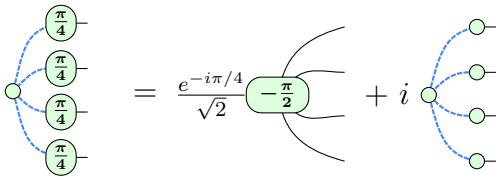
□

There are various cat state decompositions that have been found. For example, the decomposition for $|\text{cat}_6\rangle$ is



(2.14)

It has an efficiency of $\alpha \approx 0.264$. Even better, for $|\text{cat}_4\rangle$, we have the decomposition



(2.15)

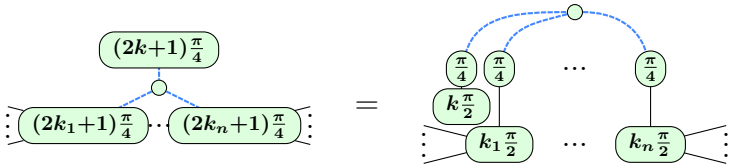
which has an efficiency of $\alpha = 0.25$.

Table 2.1 shows the best known efficiencies of the $|\text{cat}_n\rangle$ decomposition for different values of n [19].

n	3	4	5	6	7	8	9	10	11	12	13	14
terms	2	2	3	3	6	6	9	9	27	27	27	27
$\sim \alpha$	0.333	0.25	0.317	0.264	0.369	0.323	0.352	0.317	0.432	0.396	0.366	0.340

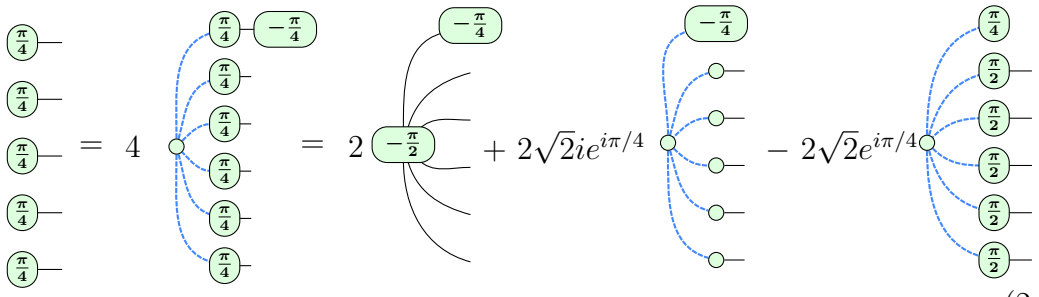
Table 2.1: $|\text{cat}_n\rangle$ decomposition

Recall that if we are dealing with the Reduced gadget form [13], any internal spider that is Clifford will be part of a non-Clifford phase gadget, by definition. Further, no two Clifford spiders will be neighbours [3]. Then, we can apply the cat state decompositions to the cat states centered at the 0-spiders that are part of these phase gadgets in the reduced gadget form, if they have ≥ 3 neighbours. This can be seen below, where $k, k_i \in \mathbb{Z}$ [3].



(2.16)

However, if the cat states are not present, we would need to fallback to a magic state decomposition. Here, the discovery of the (non-stabiliser) decomposition of 5-qubit magic states using the decomposition of $|\text{cat}_6\rangle$ gives us a guaranteed way to decompose a circuit with a T-count ≥ 5 [3].

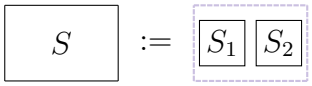


(2.17)

We then have a resulting $\alpha = \frac{\log_2(3)}{4} \approx 0.396$, which is already better than the BSS decomposition. In the next chapter, we will continue using the algorithm from [3] and improve upon the use of these cat and magic state decompositions.

2.5 Graph Cuts

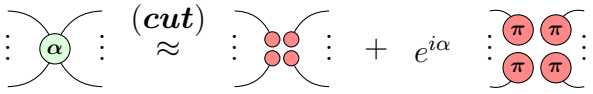
In a complementary approach to reducing the effective α of the stabiliser decomposition, graph cuts offer a different perspective [4]. Suppose we have 2 Clifford+T diagrams S_1, S_2 of T-counts t_1, t_2 that both have a stabiliser decomposition efficiency of α.



(2.18)

If we combine the diagrams into a single diagram S, we could expect the stabiliser decomposition efficiency of S to be α, coming from having $2^{\alpha(t_1+t_2)}$ stabiliser states. However, since we can treat S_1, S_2 independently and multiply their stabiliser decompositions, the actual number of stabiliser states is instead $2^{\alpha t_1} + 2^{\alpha t_2} \leq 2^{\alpha t+1}$, where $t = \max(t_1, t_2)$. This gives a resulting $\alpha' \leq \alpha \frac{t+1}{t_1+t_2}$. In the ideal case, we have that $t_1 = t_2$, so $\alpha' \leq \frac{\alpha}{2} + \frac{1}{2t}$, a vast improvement especially for a high T-count. It is with this motivation that graph cuts are another promising approach.

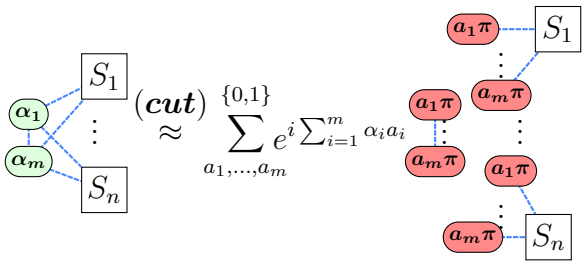
First, by definition, we can see that



(2.19)

Note that while this decomposition (***cut***) is not necessarily on a state, because of OCM, and the use of cups and caps, we can easily consider this to fit the definition of a Decomposition.

Then if we have a diagram with subdiagrams $S_1, ..., S_n$, we can see that the following holds.



(2.20)

At the cost of m cuts, and thus 2^m terms, the diagram is split into $\leq m^2 + n$ subdiagrams that can be treated independently. The $\leq m^2$ scalars in the form of spider-pairs are trivially scalars (1.10), and there are only $\leq n$ subdiagrams with T-counts > 0. If each subdiagram S_i has the T-count of t_i, we have a sum of 2^m terms, each value which can be obtained with just $\leq m^2 + \sum_{i=1}^n 2^{\alpha t_i}$ stabilizer terms. This gives a total of $\leq 2^m(m^2 + \sum_{i=1}^n 2^{\alpha t_i})$ stabilizer terms. We can obtain the efficiency.

$$\begin{aligned}
\alpha' &\leq \frac{\log_2(2^m(m^2 + \prod_{i=1}^n 2^{\alpha t_i}))}{\sum_{i=1}^n t_i} \\
&= \frac{m + \log_2(m^2 + \prod_{i=1}^n 2^{\alpha t_i})}{\sum_{i=1}^n t_i} \\
&\leq \frac{m + 2\log_2(m) + \sum_{i=1}^n \alpha t_i}{\sum_{i=1}^n t_i}
\end{aligned} \tag{2.21}$$

In the ideal case where $t_i = t$ for all i, we can produce a crude upper bound

$$\begin{aligned}
\alpha' &\leq \frac{\alpha}{n} + \frac{m + \log_2(m^2 n)}{nt} \\
&< \frac{\alpha}{n} + \frac{3m + n}{nt} \\
&= \frac{\alpha}{n} + \frac{3m}{nt} + \frac{1}{t}
\end{aligned} \tag{2.22}$$

Keeping the ratio m/n small is key to reducing the effective α. Clearly, fewer cuts are better, with preferably the end result being many disconnected subdiagrams.

Another way graph cuts also reduce the effective α need not necessarily be by obtaining disconnected subdiagrams. On pseudo-structured circuits produced from CNOTs, phase gates, and Toffolis, graph cuts were used to take advantage of the structure to optimise for the use of spider (***f***)usion in cascading fashion, where a single cut can lead to a large reduction in the T-count [5]. There, the effective efficiency obtained was $0.1 \lesssim \alpha \lesssim 0.2$. More details on this will be discussed in Section 3.3.

The same was also found for extremely structured circuits involving cat states, where a single decomposition reduced the T-count by 286, for an $\alpha \approx 0.0035$ [4], though this was an ideal case which is unlikely to occur in practice.

2.6 Subgraph Complement

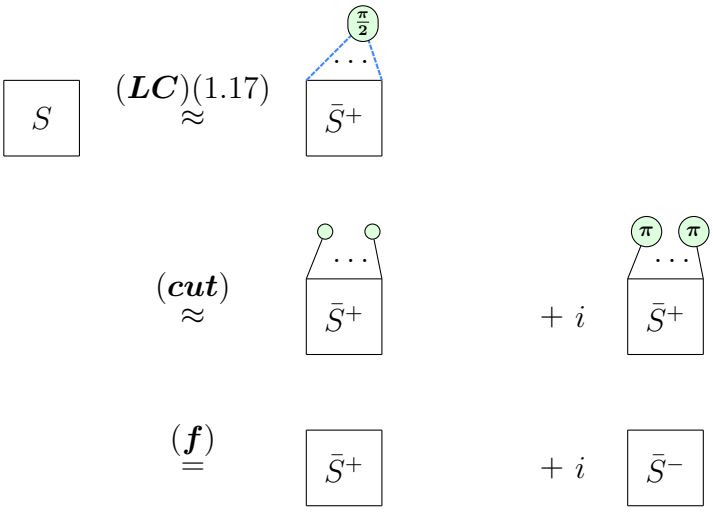
A further addition to the strength of graph cuts is the subgraph complement approach. For a graph-like ZX-diagram (V, E), with K being the set of edges for a complete graph on V, its *complement* is simply $(V, K \backslash E)$. We define a *subdiagram* to simply be a subgraph of the graph-like ZX-diagram. If we analyse ($\boldsymbol{LC}$) closely, and consider the Hopf rule ($\boldsymbol{x}$) making parallel edges cancel, there seems to be a way to obtain a subgraph complement with some manipulation of ZX-rules. In fact, we have the following result.

Proposition 2.6.1 ([4])**.** Suppose we have a subdiagram S of a graph-like ZX-diagram. Then

$$\boxed{S} \approx \boxed{\bar{S}^+} + i\boxed{\bar{S}^-} \tag{2.23}$$

where $\bar{S}^+, \bar{S}^-$ are the complements of S where the spider phases differ from S by $\pm\frac{\pi}{2}$ respectively.

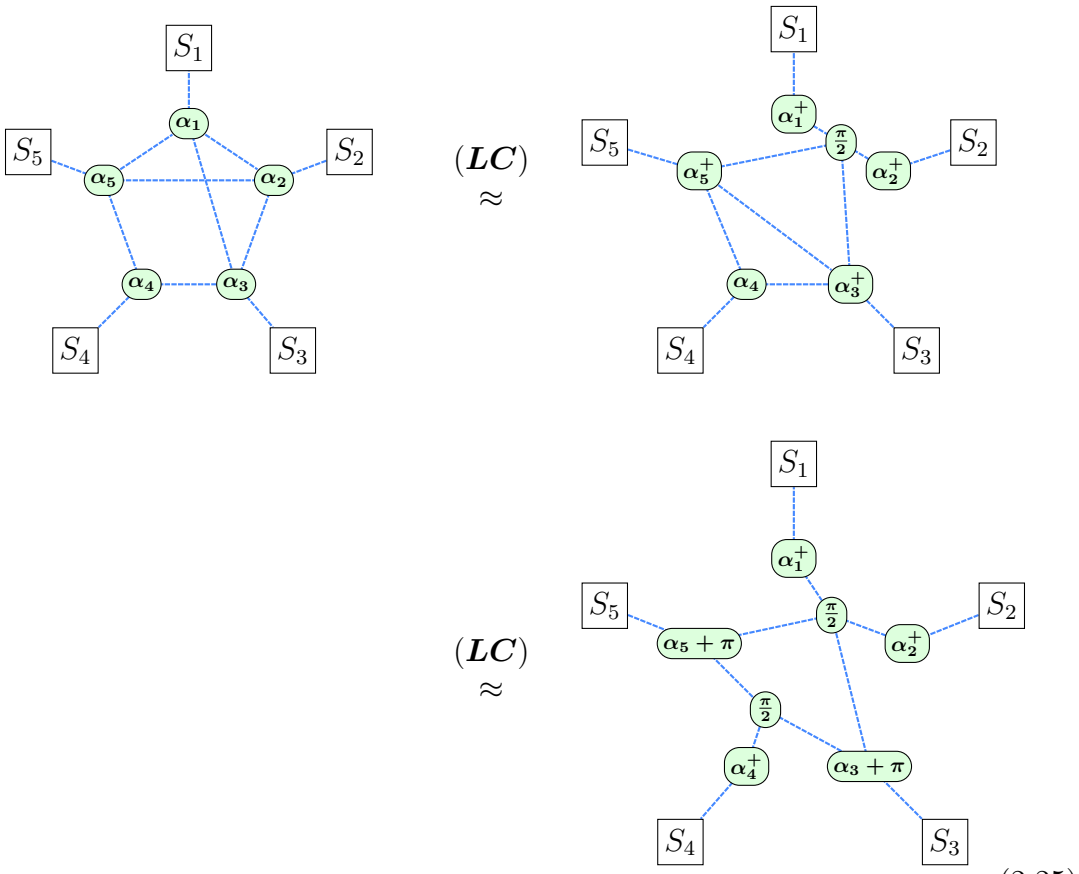
Proof. The proof is adapted from [4].



(2.24)

□

The power of this proposition can be demonstrated with the following example. Suppose we have the following graph-like ZX-diagram, where S_i are subdiagrams.



(2.25)

Following from (2.20), we can apply 5 cuts to split the diagram into 5 disconnected subdiagrams, giving us 2^5 terms. However, we can apply Prop. 2.6.1 twice to give us 2^2 terms instead.

3 Heuristics

Contents

3.1 Introduction 24
3.2 Greedy Algorithm 26
3.3 Cut Heuristic 27
3.3.1 CNOT Sandwich Heuristic 29
3.3.2 Gadget $|\text{cat}_3\rangle$ Heuristic 30
3.3.3 Lone Phase Heuristic 34
3.3.4 Paired Heuristics 34
3.3.5 Greedy Algorithm with Cut Heuristic 36
3.3.6 Other Heuristics 37
3.4 Complexity Analysis 39

This chapter details heuristics used to speedup the stabiliser decomposition process. It involves the novel conceptualisation of a cut heuristic that is valid and useful, and details the implementation of the heuristic in practice. The heuristic is closely tied with the efficiency value α, with its algorithmic implementation building off of work by Kissinger et al. [3]. The methods used by Sutcliffe and Kissinger, as well as Codsi, are also formalised using this idea of a cut heuristic [4, 5].

3.1 Introduction

The task of obtaining the stabiliser decomposition for classical simulation purposes has only been discussed at a high level so far. While we have the various methods of magic state, cat state decompositions, along with graph cut decompositions, we have not yet detailed the specific steps taken to obtain the stabiliser decompositions in the most efficient way known.

In modeling our task, we can think of the stabiliser decomposition almost as a search problem, where we are searching for the most efficient stabiliser decomposition. We can visualize an example as in Figure 3.1, where V is a Clifford+T diagram at the root. The directed edges represent the decomposition used from the parent to the child node, and the leaves d_i are the possible stabiliser decompositions.

One question that arises is how we can efficiently traverse this tree. It is strongly believed that the stabiliser rank itself must increase exponentially with the T-count

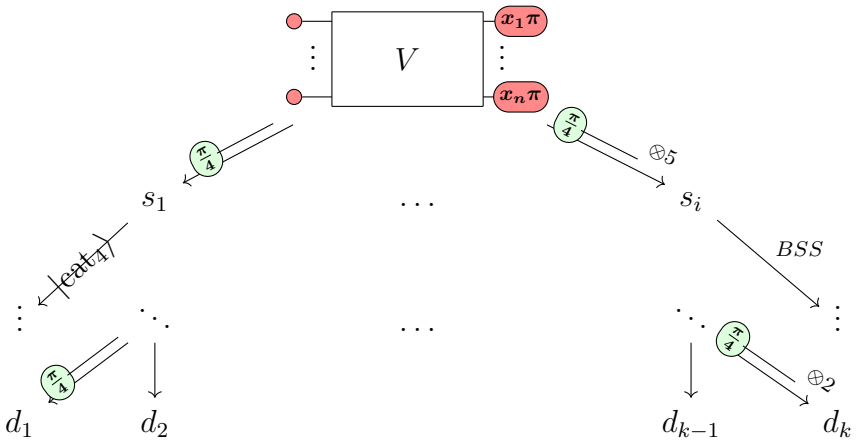


Figure 3.1: Example tree for a stabiliser decomposition

t, so we cannot have $\chi(V) = 2^{o(t)}$ [20], as cited in [14]. Thus, even if it was possible to know the most efficient decomposition, at each node, we would still face the exponential number of terms. In Figure 3.1, this means that not just that we have an exponential number of decompositions to consider, but also that every stabiliser decomposition $|d_i| = O(2^{\alpha t})$ would have an exponential number of terms.

Algorithm 3.1.1. To traverse the tree, we might take the following steps [2, 3].

1. Perform necessary spider ($\boldsymbol{f}$)usions to reduce to graph-like form.
2. Apply a chosen decomposition ("choose an edge").
3. Simplify the resulting Clifford+T diagram using ZX-simplify. Some Clifford spiders may be removed, and some T-like spiders may be combined.
4. Repeat steps 1-3 until the T-count is 0.

The complexity from using this algorithm was found to be $O(N^3 + 2^{\alpha t}t^2)$, where N is the number of gates in the original circuit [2]. Clearly, any circuit with a T-count t that is not insignificant will mean that the complexity is dominated by $O(2^{\alpha t}t^2)$.

In this thesis, we will focus on the second step of the algorithm, where we choose the decomposition to apply at each node. There are a variety of ways to choose a decomposition, detailed in the next few sections.

3.2 Greedy Algorithm

The most obvious way to obtain an efficient decomposition is to, at each step, use the decomposition that has the smallest associated value of α as possible. This is essentially a greedy algorithm, and is the method used in [2, 3]. Intuitively, this should give us a decomposition with an α upper bounded by the worst decomposition available. More formally, we have the following lemma.

Lemma 3.2.1. Let d be a stabiliser decomposition, and let $\alpha_1, ..., \alpha_n$ and $t_1, ..., t_n$ be the α values and T-count reductions of each of the n constituent decompositions to obtain d from the initial Clifford+T diagram. Then the efficiency α value of the decomposition d is given by

$$\alpha(d) \leq \frac{\sum_{i=1}^{n} \alpha_i t_i}{\sum_{i=1}^{n} t_i} \tag{3.1}$$

Proof. Each decomposition gives $\leq 2^{\alpha_i t_i}$ terms, for a total of

$$\leq \prod_{i=1}^{n} 2^{\alpha_i t_i} = 2^{\sum_{i=1}^{n} \alpha_i t_i}$$

terms. Since the final T-count is 0, the initial T-count is $\sum_{i=1}^{n} t_i$. By definition, we have

$$\alpha \leq \frac{\sum_{i=1}^{n} \alpha_i t_i}{\sum_{i=1}^{n} t_i}$$

□

This tells us that the final α value is essentially a weighted average of the α_i values, weighted by their T-count reductions. It is easy to see that if all decompositions have $\alpha_i \leq \alpha_{max}$, then (3.1) implies the overall $\alpha \leq \alpha_{max}$. The actual α value in practice is likely to be much lower than this upper bound, because of step 3 in Algorithm 3.1.1, where we simplify the diagram using ZX-simplify [2]. In [2], although the BSS decomposition has the $\alpha \approx 0.468$, the actual α value obtained in experiments was lower, at ≈ 0.4 [5].

A step further than using the α value of a decomposition, is to consider the α value when including the simplification step. In [4], this is defined as the *effective* α_e value. If the associated number of T-like spiders removed is r, r_e, where $r \leq r_e$ before and after simplification respectively, then we have

$$\alpha_e = \frac{r}{r_e}\alpha \tag{3.2}$$

This relation can make a significant difference especially for low values of r. In the next section, we also consider the structure of the diagram to increase r_e.

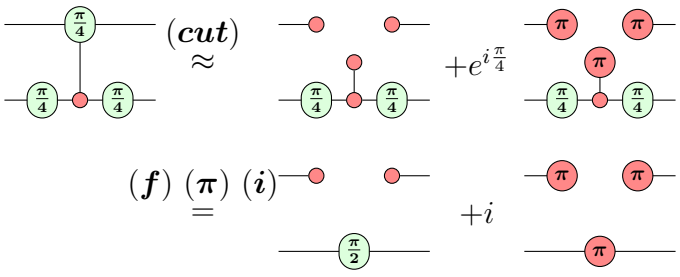
Recall also that the Algorithm 3.1.1 has $O(t^2)$ part of the complexity between each step of the decompisition. Given that we have k decompositions to choose from at each node, if we employed a brute force approach to find the best stabiliser decomposition, and attempted to find the best decomposition from a node traversing down d levels in the tree, it would take $O(k^{d+1}m^d t^2)$ time, where m is the maximum number of terms coming from a decomposition, for the k decompositions to be applied to all the m terms after each level. For example, the above approach of obtaining α_e takes $d = 0$.

In theory, this could be extended to larger values of $d \geq 1$, but with $km \geq 2$, this would mean that the overall exponential complexity of $O(2^{\alpha t + d \log mk} t^2)$ would be affected. This motivates the idea of using heuristics instead.

3.3 Cut Heuristic

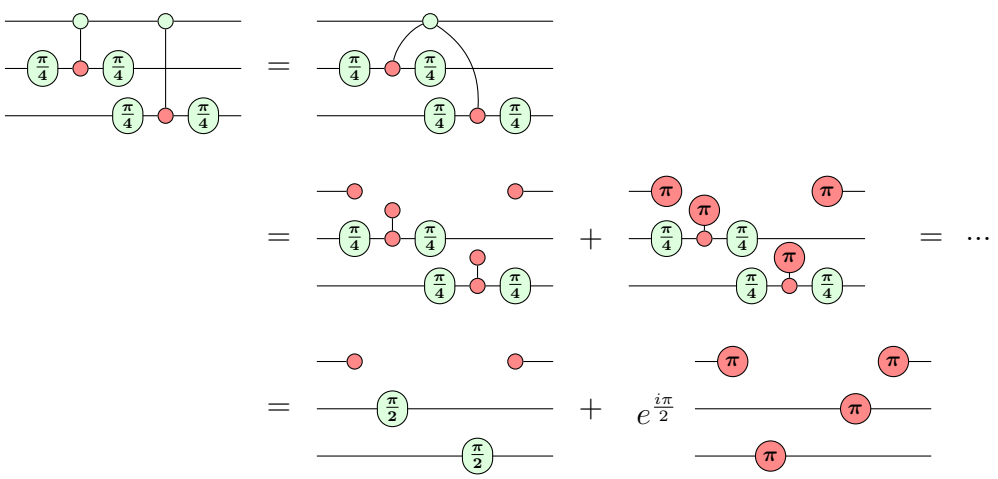
A disadvantage of the greedy algorithm is that it does not take into account the structure of the diagram. When we only consider the α value of the decomposition, we fail to take into account that T-like spiders can be fused in between decompositions where decompositions with seemingly higher α values may actually be more efficient.

In [5], the heuristic used considers the structure of CNOT gates placed in between T gates. The following observation was made.



(3.3)

Using the graph (***cut***), which without simplification has $\alpha = 1$ if the spider being cut is a T-like spider, we can instead obtain a decomposition with $\alpha \approx 0.333$, with simplication, for the reduction of 3 T-like spiders at the cost of 2 terms. In fact, this structure can be stacked in the following way.



(3.4)

Using these observations, a heuristic can be developed, where the main idea is to count the number of T-like spiders that can be removed, divided by how many cuts are needed to remove them.

Definition 3.3.1. Let V be a Clifford+T diagram, and let v be a vertex in V. Let C be the set of sequence of graph cuts. For each $c \in C$ where there are $|c| = n$ cuts in the sequence, let $t(c)$ be the number of T-like spiders removed by the sequence of cuts using (***cut***), including simplifications between each cut. Then a *cut heuristic* for the sequence c of length n is defined as any heuristic $h : V \to \mathbb{R}^+$ such that

$$\exists c = (v, ..., v_n) \in C, 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v) \tag{3.5}$$

where $h_e(v)$ is the effective heuristic value of c. ▲

For the cut heuristic to be valid, it should not overestimate the effective heuristic value h_e. For the cut heuristic to be useful, it should be close to the value of h_e. We have that a higher cut heuristic value implies a more efficient decomposition. In fact, the associated α_e value of the sequence of decompositions is given by

$$\alpha_e(c) = \frac{t(c)}{|c|} = \frac{1}{h_e(v)} \leq \frac{1}{h(v)} = \alpha_h(c) \tag{3.6}$$

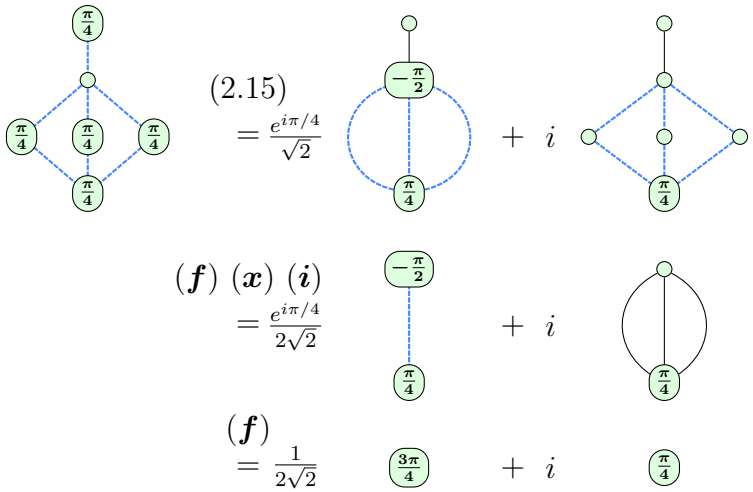
where $\alpha_h(c)$ is the associated α value of the cut heuristic, and since the $n = \log_2(2^n)$ cuts contribute to 2^n terms.

If the sequence of cuts leads to a stabiliser decomposition, the effective cut heuristic is closely related to our α_χ value, but does not take into account the other decompositions we have available. For some Clifford+T diagram V, let C be the set of sequences of decompositions resulting in a stabiliser decompositions that only

use graph cuts, and D be the set of stabiliser decompositions. Let $f : C \to D$ be the function that maps a sequence of cuts to the decomposition obtained. Let $c^* = (v^*, ...) \in C$ be the most efficient decomposition using graph cuts such that

$$|f(c^*)| = \min\{|f(c)| \mid c \in C\} \tag{3.7}$$

Here, $\forall c = (v, ...) \in C, h_e(v^*) \geq h_e(v)$. If c^* results in a decomposition where $|f(c^*)| = \chi(V)$, then $\alpha_\chi = \alpha_e(c^*)$ [1]. In general, however, $\alpha_\chi \leq \alpha_e(c^*)$. A simple example of this can be seen in the following diagram, where the decomposition makes use of $|\text{cat}_4\rangle$ (2.15). It starts with a T-count of 5.



(3.8)

Using only cuts would require 2 of them, which would give 2^2 terms instead of 2 terms. We have

$$\alpha_\chi \leq \frac{1}{5} < \alpha_e(c^*) = \frac{2}{5} \tag{3.9}$$

3.3.1 CNOT Sandwich Heuristic

In [5], the cut heuristic used concentrated on sequences of cuts that were looking to eliminate CNOT gates sandwiched between T-like spiders gates. As a result, 71% of the time, they were able to obtain c^* on small circuits. Ultimately, efficiency values of $0.1 \lesssim \alpha \lesssim 0.2$ were obtained in practice, considering semi-structured diagrams.

Using the cut heuristic means that there is some expert knowledge on the sequences of cuts to be made. With better consideration of sequences, we can continue to use the greedy approach on α values, with the hope of obtaining a more efficient decomposition. In the tiered approach of [5], a brief description is that increasing lengths of sequences were considered, where each sequence of cuts c_i for

[1] Here, and in (3.6), we overload notation by equating $\alpha(f(c^*)) = \alpha(c^*)$

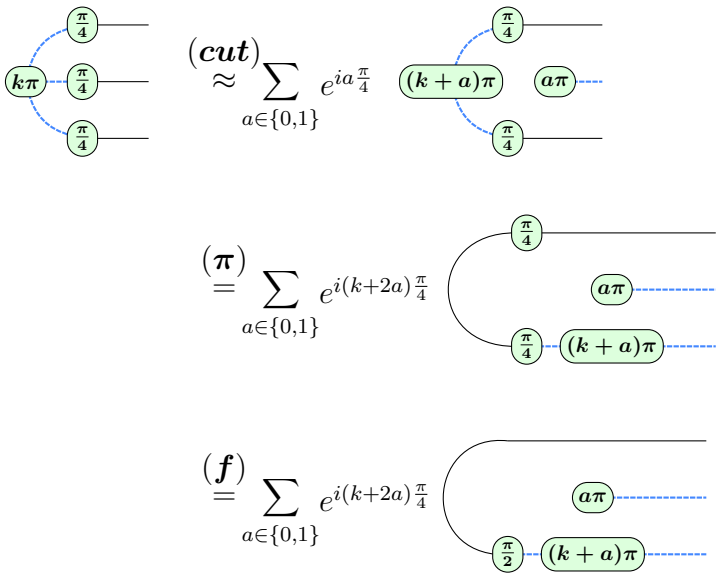
tier i led to computation of the cut heuristic $h_{i+1}(v)$, corresponding to sequences $c_{i+1} = k_i : c_i$ for tier $i+1$ for some sequence of vertices $k_i = (v, ...)$. Eventually, the algorithm terminates when there is some $l = i_{\max}$ where there are no sequences of vertices k_l to compute c_{l+1} at tier $l+1$. The result is the computation of h_i values for tiers $i \leq l$, and the choice to cut v where

$$v_{best} := \arg\max_v h_l(v) = \arg\min_v \alpha_{h_l}(v) \tag{3.10}$$

3.3.2 Gadget $|\text{cat}_3\rangle$ Heuristic

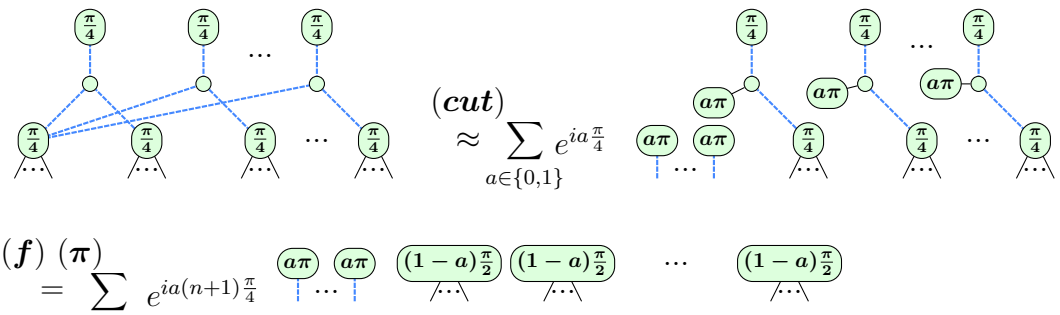
The cut heuristic is also useful outside of CNOT sandwiches. Particularly, the reduced gadget form contains structures that when cut, reduce the T-count greatly.

In [4], the following observation was made when applying the cut to a $|\text{cat}_3\rangle$.



(3.11)

The associated efficiency is $\alpha = 1/3$ here, from removing 3 T-like spiders at the cost of 1 cut. This means that any T-like spider that is part of multiple $|\text{cat}_3\rangle$ can contribute to removing all the associated T-like spiders at once. A visualisation can be seen below (3.12). It can be noted that a $|\text{cat}_3\rangle$ and a two-legged phase gadget are interchangeable concepts in the reduced gadget form.



(3.12)

In [4], the effective α_e value was used, after determining the number of $|\text{cat}_3\rangle$ each T-like spider was part of, in order to find the most suitable cut to compare with the other decompositions. As such, there was no calculation of the heuristic α_h value. We will see in subsection 4.1.2 as well, how this observation was related to the improved bounds on the complexity of simulating IQP circuits [6].

Here, we can define the heuristic as follows.

$$h(v) = 1 + 2 \cdot (\#\ |\text{cat}_3\rangle\ v \text{ is part of}) \tag{3.13}$$

since each neighbouring $|\text{cat}_3\rangle$ can contribute to the removal 2 T-like spiders.

Proposition 3.3.2. For any ZX-diagram in reduced gadget form, the heuristic $h(v)$ (3.13) is a valid cut heuristic for a single cut if v is T-like.

$$\forall v \in V, v \text{ is T-like} \implies 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v)$$

Proof. If v is part of a phase gadget, this is trivial, as $h(v) = 3$. Suppose v has neighbours $w_1, ..., w_k$ that are 0-spiders that are part of $|\text{cat}_3\rangle$ states. Note that $w_1, ..., w_k$ are all part of phase gadgets, by Definition 1.4.5. Each w_i then has 2 legs, one of which is connected to v. Since no two phase gadgets have the same exact legs, we must have for all $w_i, w_j, i \neq j$, that their other legs do not connect to the same vertex u. By cutting only v, we can remove at least $2k + 1$ T-like spiders corresponding to k $|\text{cat}_3\rangle$ states, and vertex v. Then we must have

$$h_e(v) \geq 2k + 1 = h(v) \tag{3.14}$$

□

We can note that even if k is small, the heuristic can be quite effective. For instance, with $k = 2, \alpha_h = 0.2$ which is already an improvement over all the $|\text{cat}_n\rangle$ decompositions in [2, 3], the best of which is $\alpha = 0.25$ for the $|\text{cat}_4\rangle$

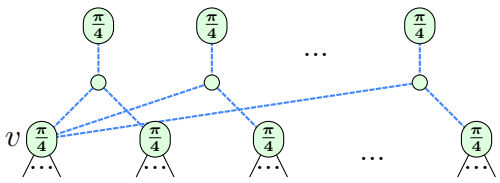


Figure 3.2: Diagram to showcase the heuristic (3.13)

decomposition. A T-like spider simply has to be connected to 2 $|\text{cat}_3\rangle$ states to improve the decomposition.

More generally, it was also considered that cutting T-like spiders part of $|\text{cat}_n\rangle$ states would convert them to $|\text{cat}_{n-1}\rangle$ states. However, this was not considered in the heuristic of [4], as they focused on obtaining the single-step α_e value.

The heuristic considering $|\text{cat}_n\rangle$ states could then be defined as follows.

$$h(v) = \sum_{i=1}^{k} \frac{2}{n(w_i) - 2} \tag{3.15}$$

where w_i are neighbours of v that are 0-spiders that are part of $\left|\text{cat}_{n(w_i)}\right\rangle$ states, for $n : V \to \mathbb{N}$. The heuristic considers sequences of cuts for every $\left|\text{cat}_{n(w_i)}\right\rangle$ that v is part of, and attempts to add up each pair of T-like spiders that could be removed with $n(w_i) - 2$ cuts. Unfortunately, this heuristic is not a valid heuristic in general.

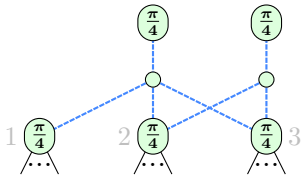


Figure 3.3: Diagram where the heuristic (3.15) is not valid

Here, the heuristic would give $h(v_2) = \frac{2}{1} + 2 = 3$, but cutting 2 vertices to remove 5 T-spiders would mean $h_e(v_2) = \frac{5}{2} < 3$.

We show that with an added condition, the heuristic is valid.

Proposition 3.3.3. Suppose v has 0-spider neighbours $w_1, ..., w_k$ that are part of all $\left|\text{cat}_{n(w_i)}\right\rangle$ states. Let $T(w_i)$ be the set of T-like spiders that is part of each $\left|\text{cat}_{n(w_i)}\right\rangle$ state, so $|T(w_i)| = n(w_i)$. Let $r = \frac{\max\{n(w_i)\}_i - 2}{\min\{n(w_i)\}_i - 2}$. Then for the heuristic $h(v)$ (3.15), we have

$$\left|\bigcup_{i=1}^{k} T(w_i)\right| \geq 2k^2 r \implies \exists c = (v, ...) \in C, 0 < h(v) \leq \frac{t(c)}{|c|} = h_e(v) \tag{3.16}$$

In other words, if the number of distinct T-like spiders part of all the $\left|\text{cat}_{n(w_i)}\right\rangle$ states is at least $2k^2r$, then the heuristic $h(v)$ is a valid cut heuristic.

Proof. Note that $w_1, ..., w_k$ are all part of phase gadgets, by Definition 1.4.5. By cutting at most $\sum_{i=1}^{k}(n(w_i)-2)$ T-like spiders as legs of the phase gadgets, we can remove at least $\left|\bigcup_{i=1}^{k} T(w_i)\right|$ T-like spiders. Then we must have

$$h_e(v) \geq \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i)-2)} \tag{3.17}$$

Then

$$\begin{aligned} h(v) &= \sum_{i=1}^{k} \frac{2}{n(w_i)-2} \\ &\leq \sum_{i=1}^{k} \frac{2}{\min\{n(w_i)\}_i - 2} \\ &= \frac{2kr}{\max\{n(w_i)\}_i - 2} \\ &= \frac{2k^2r}{k(\max\{n(w_i)\}_i - 2)} \\ &\leq \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i)-2)} \\ &\leq h_e(v) \end{aligned} \tag{3.18}$$

□

The proposition condition is generally unlikely to be satisfied, and ultimately leads to the heuristic performing poorly in general. From the proof, we can gather that there is in fact a cut heuristic that is valid.

Corollary 3.3.4. The following heuristic is a valid cut heuristic.

$$h(v) = \frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{\sum_{i=1}^{k}(n(w_i)-2)} \leq h_e(v) \tag{3.19}$$

Unfortunately, although the heuristic is valid, it can vastly underestimate the effective heuristic value. This happens when there is a large overlap in T-like spiders connected to different phase gadgets. A simple example is as follows, where we have $|\text{cat}_n\rangle$ for $n = 3, 4, 5$.

$h_e(v_1) \geq \frac{7}{3}$ since only 3 cuts are needed to remove 7 T-spiders, but $h(v_1) = \frac{4}{3+2+1} = \frac{2}{3} < \frac{7}{3}$.

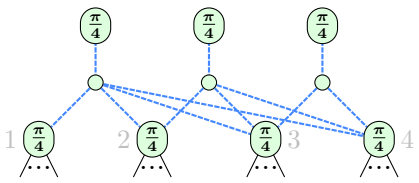
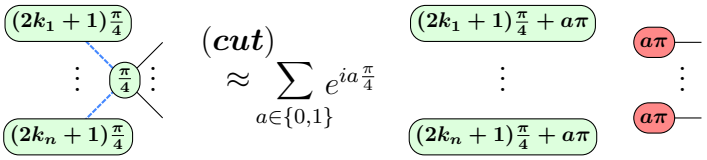


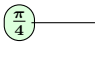
Figure 3.4: Diagram where the heuristic (3.19) underestimates the effective heuristic value

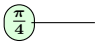
3.3.3 Lone Phase Heuristic

Another heuristic turns out to be somewhat common to measure is the lone phase heuristic. We can observe the following decomposition.

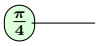


$$(\boldsymbol{cut}) \approx \sum_{a\in\{0,1\}} e^{ia\frac{\pi}{4}} \tag{3.20}$$

Since the legless spiders can easily be treated as scalars, a T-like spider connected to n magic states  would allow the T-count to drop by n. We define the *lone phase heuristic* as follows.

$$h(v) = 1 + \#\ \text{  } v \text{ is connected to} \tag{3.21}$$

It's clear to see that this is a valid cut heuristic.

What we have then is that we can combine the two "simple" heuristics (3.13, 3.21) so far. If n is the number of $|\text{cat}_3\rangle$ states v is part of, and m is the number of magic states  v is connected to, then we have the following heuristic.

$$h(v) = 1 + 2n + m \tag{3.22}$$

There is clearly no overlap between the two types of T-like spiders that each heuristic considers, so the heuristic is still valid. Surprisingly, just a simple combination of the two heuristics can be quite effective in practice, as we will see in the next chapter.

3.3.4 Paired Heuristics

The heuristics thus far concentrate on cutting single vertices, with the hope of further simplification aftewards. However, we can also consider heuristics that involve cutting pairs of vertices. Consider the following diagram.

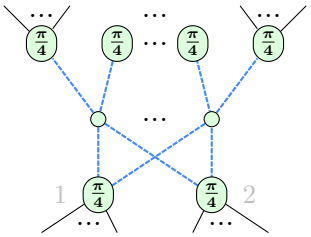
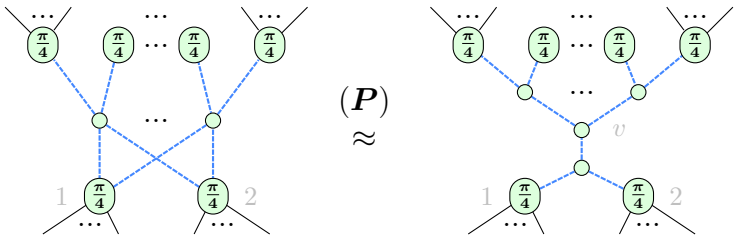


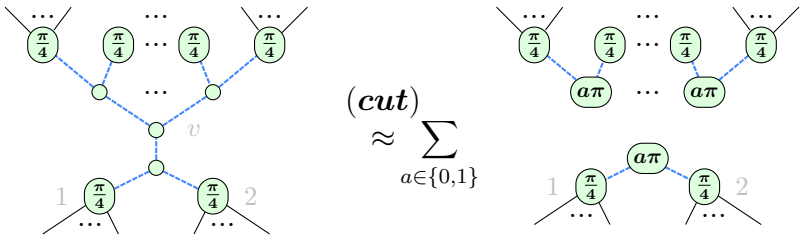
Figure 3.5: A pair of vertices connected to 3-legged phase gadgets

We see at the top that we have n 3-legged phase gadgets (a.k.a. $|\text{cat}_4\rangle$ states), fully connected to a pair of T-like spiders. At first glance, it might seem like gadget fusion (***GF***) can almost be applied, and that we need to remove the n T-like spiders at the top to do so. However, this is not too optimal, since we at most remove $2n$ T-like spiders with n cuts, giving an $\alpha = 0.5$. Instead, we can exploit the fully connected aspect of the pair of T-like spiders, by considering the (***P***)ivot rule again.

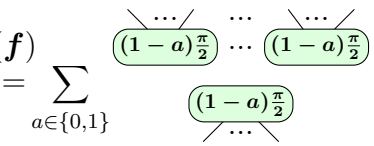


(3.23)

Then we can (***cut***) vertex v to remove the T-like spiders.

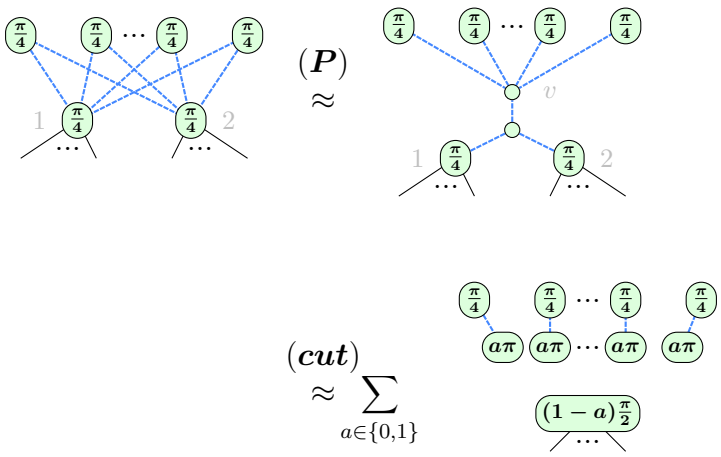


(3.24)



A total of $2n + 2$ T-like spiders are removed with a single, giving an $\alpha = \frac{1}{2n+2}$.

We can go even further, if we draw inspiration from the lone phase heuristic (3.21). If we have a pair of T-like spiders fully connected to m 2-legged T-like spiders, we can again use the (***P***)ivot rule to our advantage.



(3.25)

We can summarise the paired heuristic as follows. Suppose v, w are T-like spiders fully connected to n 3-legged phase gadgets, and m 2-legged T-like spiders respectively. Then we have two heuristics.

$$\begin{aligned} h_1(v,w) &= 2 + 2n \\ h_2(v,w) &= 2 + m \end{aligned} \tag{3.26}$$

In this case, we don't necessarily merge the two heuristics since the pivot vertices are different. We could consider a combined heuristic, where the 2 pivots happen at the same time, so 2 cuts are needed to remove all the vertices.

$$\begin{aligned} h(v,w) &= \frac{2 + 2n + m}{2} \\ &< \frac{h_1(v,w) + h_2(v,w)}{2} \\ &\leq \max\{h_1(v,w), h_2(v,w)\} \end{aligned} \tag{3.27}$$

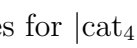
This is essentially the average of the two heuristics, which is necessarily smaller than the maximum between the two heuristics. Since we can process the heuristics sequentially, we can just consider them separately as before.

3.3.5 Greedy Algorithm with Cut Heuristic

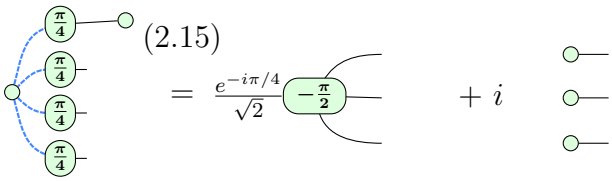
A natural extension of the greedy algorithm is to also consider the cut heuristic on top of the cat decompositions.

We have the α values from decompositions of $|\text{cat}_4\rangle$, $|\text{cat}_6\rangle$, $|\text{cat}_5\rangle$, $|\text{cat}_3\rangle$, and magic state $\frac{\pi}{4}$ $^{\otimes 5}$ being $0.25 < 0.264 < 0.317 < 0.333 < 0.396$ respectively. Depending on the value of the best heuristic (3.22), $\max_v h(v)$, and the best available $|\text{cat}_n\rangle$ decomposition in the diagram, we can continue to use the greedy approach. We have the following simple algorithm.

Algorithm 1 Greedy Algorithm with Heuristic

Input: α values for $|\text{cat}_4\rangle$, $|\text{cat}_6\rangle$, $|\text{cat}_5\rangle$, $|\text{cat}_3\rangle$, and  , and the graph G
Output: Chosen state and corresponding decomposition
Let $\alpha_4 = 0.25, \alpha_6 = 0.264, \alpha_5 = 0.317, \alpha_3 = 0.333, \alpha_{\otimes 5} = 0.396$
Compute the best catstate $|\text{cat}_{best}\rangle$ in G with the lowest α value
Let α_{best} be the α value of $|\text{cat}_{best}\rangle$, or of if no $|\text{cat}_{best}\rangle$ is found
Compute α_h on G using a heuristic
if $\alpha_h < \alpha_{best}$ **then**
    Use the decomposition corresponding to the heuristic
else
    Use the decomposition corresponding to $|\text{cat}_{best}\rangle$ or 
end if

Even with a small values of n, m, the heuristic can be quite effective. Just having $n = m = 1$ can already give a $\alpha_h = 0.25$, matching the best $|\text{cat}_4\rangle$ decomposition. Furthermore, this will cut the graph at a vertex, as compared to the $|\text{cat}_4\rangle$ decomposition, which only locally partitions the second of its 2 terms, while its first term remains connected, seen below.



(3.28)

This heuristic also works in a diagram without any phase gadgets where $n = 0$, concentrating on the T-like spiders. In this scenario, having $m = 2$ will already give a better $\alpha_h = 1/3$ than the default 5-qubit magic decomposition at $\alpha \approx 0.396$.

3.3.6 Other Heuristics

Another heuristic considered, based off (3.19) takes into account the overlap of T-like spiders, and not overcount the number of cuts needed. Suppose the T-spider part of each phase gadget w_i is labelled vertex g_i. Each phase gadget as part of a $|\text{cat}_n\rangle$ would only need a maximum of $n - 2$ cuts, so our heuristic would look like

$$\frac{\left|\bigcup_{i=1}^{k} T(w_i)\right|}{1 + \left|\bigcup_{i=1}^{k} T(w_i) - \{g_i, v, u_i\}\right|} \tag{3.29}$$

where $\forall i, u_i \in T(w_i)$ is a chosen T-like spider on the leg of phase gadget w_i, that does not need to be cut. We would want to choose the u_i that minimizes the denominator, so we have the following heuristic.

$$h(v) = \max_{\forall i, u_i \in T(w_i)} h(v) \tag{3.30}$$

where the denominator is minimized.

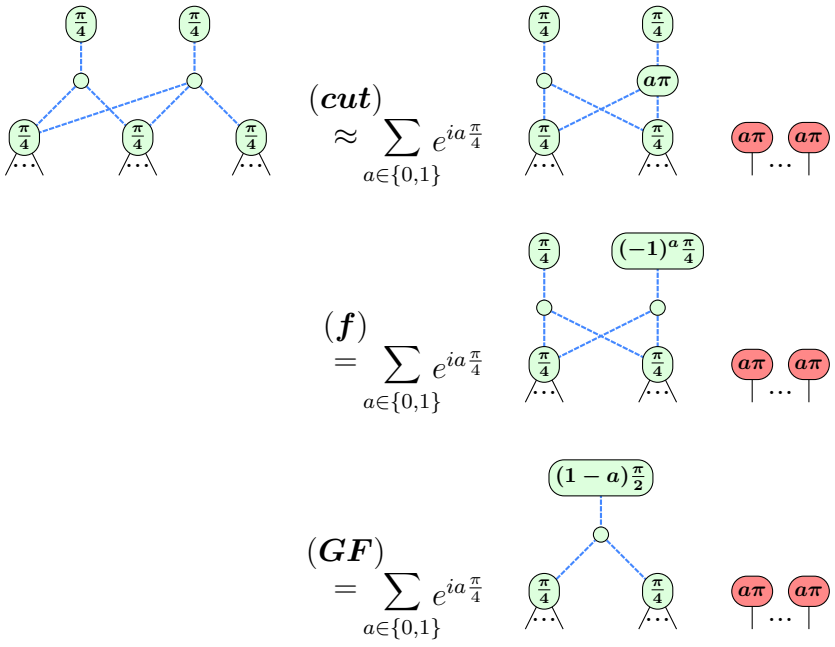
$$t_{\min} = 1 + \min_{\forall i, u_i \in T(w_i)} \left| \bigcup_{i=1}^{k} T(w_i) - \{g_i, v, u_i\} \right| \tag{3.31}$$

It is easy to see that this heuristic is valid, by the same argument as the previous Proposition 3.3.3. Solving this is akin to maximum bipartite matching. The is because to minimize the number of cuts, the number of T-like spiders matched with a phase gadget needs to be maximized, while only one T-like spider can be matched with each phase gadget. The matching can be found in polynomial time using any maximum bipartite matching algorithm like the Hopcroft-Karp algorithm [21]. Unfortunately, this heuristic is not as effective as the previous heuristic (3.13) just involving $|\text{cat}_3\rangle$ states.

The phase gadget heuristic is also considered, where the T-like spiders connected to the legs of two phase gadgets w_i, w_j are compared. Let $T(w_i), T(w_j)$ be the T-like spiders connected to the legs of w_i, w_j respectively. Recall from (***GF***), that gadgets with the same set of legs can be fused. If all the T-like spiders from the symmetric difference $T(w_i) \Delta T(w_j)$ are removed, where

$$A \Delta B = (A \cup B) - (A \cap B) \tag{3.32}$$

then the T-count can be reduced by 2 via gadget fusion (***GF***), as seen below.



(3.33)

We could then have the following heuristic, for the simplest case where

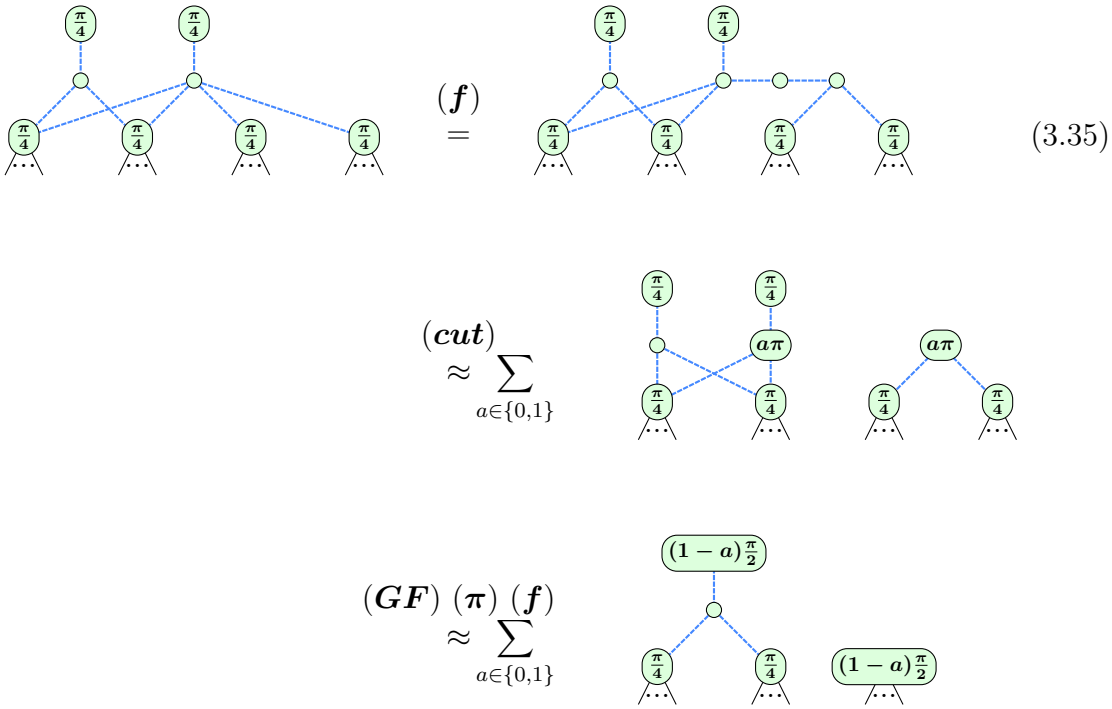
$$\exists i, j, T(w_i)\Delta T(w_j) = \{g_i, g_j, v\}$$

$$h(v) = 1 + 2 \cdot |\{\{w_i, w_j\} \mid T(w_i)\Delta T(w_j) = \{g_i, g_j, v\}\}| \tag{3.34}$$

where g_i, g_j are the T-like spiders part of the phase gadgets w_i, w_j respectively.

This heuristic is valid, due to the requirement that $\{w_i, w_j\}$ must be distinct. As such, there is no overcounting of the number of T-like spiders that can be removed.

Similarly, the paired version of the heuristic can be considered, where there are 2 extra T-like spiders preventing gadget fusion.



(3.35)

Alas, these phase gadget heuristics are not effective in practice, and earn their place in the "other heuristics" category. Scenarios for being close to gadget fusion are not common enough, and tend to take more cuts than necessary, by which time the cut heuristics (3.22, 3.26) would have already been more effective.

3.4 Complexity Analysis

Here, we consider the complexity of computing the heuristics.

Suppose we have a reduced gadget form V with n phase gadgets and m T-like spiders not part of phase gadgets. Let $d(v)$ be the degree of a vertex v.

For the heuristic (3.13), it is easy to compute the heuristic in $O(mn)$ time. Checking which 0-spiders are part of $|\text{cat}_3\rangle$ states can be done in $O(n)$. Each T-like spider v can be processed in $O(d(v)) = O(n)$ time, and checking for all of them takes $O(mn)$, for an overall of $O(n + mn) = O(mn)$ time.

The heuristic (3.15) has a similar complexity of $O(mn)$.

For the $|\text{cat}_n\rangle$ heuristic (3.19), a single T-like vertex v, and its 0-spider neighbours $w_1, ..., w_k$, it takes

$$O\left(d(v) + \sum_{i=1}^{k} d(w_i)\right) = O(n + mn) = O(mn) \tag{3.36}$$

time to compute the heuristic. For all m T-like spiders, this takes $O(m^2 n)$ time.

The lone phase heuristic (3.21) can be computed in $O(m^2)$ time, since each T-like spider can be processed in $O(m)$ time, for all m T-like spiders.

Combining the complexities for the general heuristic (3.22) will take $O(mn+m^2)$ time. With the existing greedy algorithm from [2, 3] that has complexity $O(2^{\alpha t} t^2)$, and since $m + n = t$, the complexity remains unchanged. Specifically, looking at the $O(t^2)$ operations in between decompositions,

$$O(t^2 + mn + m^2) = O(t^2) \tag{3.37}$$

Comparing to the method of [4] which requires computing α_e values (3.2) for each of the k decompositions considered to select the best one, the number of operations between each decomposition there is $O(kt^2)$.

The paired heuristics are (3.26) are slightly more costly than the single heuristics, with a complexity of $O(m^2(n + m))$. Each pair of T-like spiders can have at most $O(n)$ phase gadget neighbours, and $O(m)$ T-like spider neighbours, and so can be processed in $O(n+m)$ time. With the $O(m^2)$ pairs of T-like spiders, the complexity is $O(m^2(n+m))$. This overhead cost comes into play since $O(m^2(n+m)) = O(t^3)$ in the worst case, compared to the original $O(t^2)$ complexity. With a higher T-count, and the reduction of α due to more efficient cuts, the paired heuristics can still be effective in practice, as we see in the next chapter.

For the heuristic taking into account the overlap of T-like spiders (3.30), the Hopcroft-Karp algorithm has a complexity of $O(mn\sqrt{m+n})$ for each T-like spider, for a total of $O(m^2 n\sqrt{m+n})$ time.

For the phase gadget heuristic, to compare two phase gadgets is $O(m)$, for a total of $O(mn)$ for all phase gadgets. Computation for all m T-like spiders can be done at the same time, so the complexity is $O(mn)$.

4 Results

Contents

4.1 Benchmarking 41
4.1.1 Random Clifford+T 41
4.1.2 Random IQP 43
4.1.3 Modified Hidden Shift 46
4.1.4 Random Clifford+T with CCZ 47
4.2 Experiments 48
4.2.1 Random Clifford+T 48
4.2.2 Random IQP 48
4.2.3 Modified Hidden Shift 50
4.2.4 Random Clifford+T with CCZ 50

This chapter presents the classes of circuits used to benchmark our heuristics, which are not novel themselves, but are used to compare the effectiveness of our methods. There are some novel analyses or generalisations of the structures of some circuits, notably the Random IQP and the Modified Hidden Shift circuits.
In the section afterward, we present the results of the benchmarking of our heuristics, with experimentation setup detailed. We also discuss the reasons behind effectiveness of heuristics to certain classes of circuits.

4.1 Benchmarking

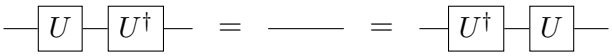
To determine the effectiveness of our heuristic methods thus far, we need to benchmark them against known methods. In this chapter, we introduce the various benchmarking methods we use to evaluate our heuristics. This is based off of various previous work, particularly those using ZX-calculus [2–5]. We will be using QuiZX [22], with features added from [3], especially the $|\text{cat}_n\rangle$ decompositions. QuiZX itself was a Rust port of PyZX, originally built in Python [23].

4.1.1 Random Clifford+T

Random Clifford+T circuits coming from *exponentiated Pauli unitaries* were used in the benchmark of the BSS decomposition [13], and subsequently continued use in

the benchmark of the $|\text{cat}_n\rangle$ decomposition [3]. The operators in this form naturally arise in the literature, including Clifford+T circuit representations [24], as cited in [13]. We introduce their definitions, pertaining to the ZX-calculus.

Definition 4.1.1 (Unitary)**.** A unitary U is a map of the form



(4.1)

▲

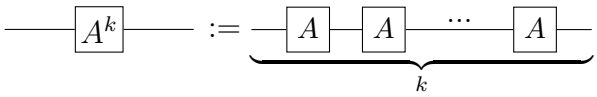
Definition 4.1.2 (Pauli exponential [11])**.** A *Pauli exponential* is any unitary of the form

$$U := e^{\pm i\theta P_1 \otimes \cdots \otimes P_n} \tag{4.2}$$

where P_i are Paulis, and $\theta \in \mathbb{R}$ is a phase, and the exponential is defined as

$$e^A := \sum_{k=0}^{\infty} \frac{A^k}{k!} \tag{4.3}$$

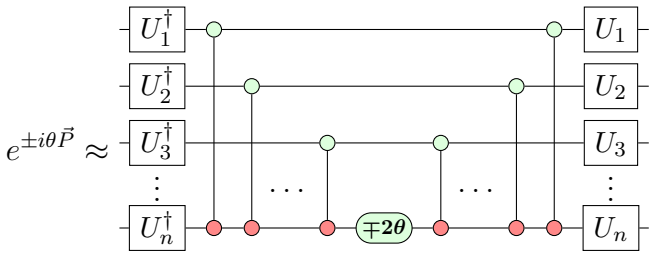
for any map A, where A^k is



(4.4)

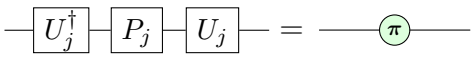
▲

Proposition 4.1.3 ([11])**.** Let $\vec{P} = P_1 \otimes \cdots \otimes P_n$ be any Pauli string with no trivial entries, so $\forall i, P_i \in \{X, Y, Z\}$. Then the Pauli exponential $e^{i\theta\vec{P}}$ is a ZX diagram where



(4.5)

Here $U_j \in \{$  $\}$ such that



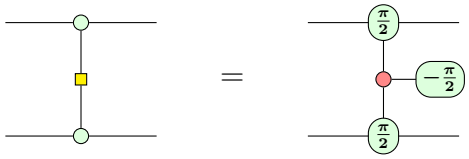
(4.6)

Choosing $\theta = (2k+1)\frac{\pi}{4}$ for $k \in \mathbb{Z}$, we can generate a random Clifford+T circuit with a fixed depth by composing these exponentiated Pauli unitaries. This fixes the T-count of the circuit, and can be used to benchmark decompositions.

4.1.2 Random IQP

Instantaneous Quantum Polynomial (IQP) circuits were used in the benchmark of [4]. These circuits were chosen as a possible model to demonstrate quantum supremacy [25]. They can likely by build in a quantum computer, while at the same time, they are a class of commuting quantum computations which are shown to be hard to simulate classically, even when adding in the presence of physically motivated constraints, such as sparsity and noise [25]. First, we introduce the CZ-gate.

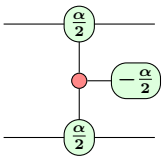
Definition 4.1.4 (CZ-gate)**.** The CZ-gate is the controlled Z-gate, defined as

$CZ :=$  (4.7)

▲

More generally, we can define the controlled Z(α)-gate, where $\alpha = k\frac{\pi}{2}, k \in \mathbb{Z}$, so $CZ(\pi) = CZ$.

Definition 4.1.5 ($CZ(\alpha)$-gate [7])**.** The controlled Z(α)-gate is the phase gadget,

$CZ(\alpha) :=$  (4.8)

▲

Definition 4.1.6 (IQP circuit [6, 26])**.** An IQP circuit is a circuit composed of Hadamard gates, $CZ(\frac{k\pi}{2})$-gates, and T^m-gates, where $k, m \in \mathbb{Z}$. ▲

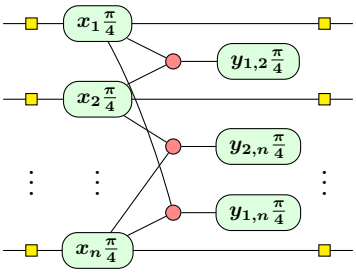


Figure 4.1: An IQP circuit as a ZX-diagram, for $x_i, y_{i,j} \in \mathbb{Z}$, as defined in [6]

Intuitively, the commutative nature of the phase gadgets can be seen directly in the ZX-calculus. We also see the presence of many phase gadgets which only have 2 legs, which lends itself directly to the strength of our heuristic (3.22), considering the number of $|\text{cat}_3\rangle$ states (3.13).

In [6], dense IQP circuits were found to be simulable in $O(\frac{\log^2 n}{n}2^n)$ time, where n is the number of qubits, which is a significant improvement and faster than $O(2^n/poly(n))$ for any $poly(n)$, compared to $O(n^4 2^{O(n^2)})$ of just using stabiliser decomposition. They concluded that current hardware is probably not enough to demonstrate quantum supremacy using IQP circuits. We note, however, that the heuristic approach can easily give us a better upper bound than than the general one.

Lemma 4.1.7. Given a dense n-qubit IQP circuit, the heuristic approach to stabiliser decomposition can be completed in $O(2^n)$ time.

Proof. Note that as stated in [6], we can decompose the IQP circuit into $O(2^n)$ just by cutting the T-like spiders corresponding to the n qubits. It suffices to show that the heuristic will pick the correct cut decompositions, or result in an equivalent number of terms. First, note that we can set the algorithm 3.1.1 to skip the ZX-simplify [2] step between each cut. Consider a T-like spider connected to a leg of a phase gadget. Note that there are only n of such spiders. We know it will always be connected to $k \geq 1$ phase gadgets (i.e. the x_i T-spiders in Figure 4.1). Also, the T-like spiders in the phase gadgets themselves are only connected to a single phase gadget by definition (i.e. the $y_{i,j}$ T-spiders in Figure 4.1). If $k = 1$ for both legs of the phase gadget, it doesn't matter which T-like spider we pick, as they would be equivalent cuts. Otherwise, if $k > 1$ for some leg, the heuristic will always pick that T-like spider, and since its $\alpha_h \leq 0.2 < 0.25$ is better than any of the $|\text{cat}_n\rangle$ decompositions, it will be picked over them in the greedy algorithm. If we are left with T-like spiders where all are connected to $k = 1$ phase gadgets, then since the only cat state present is the $|\text{cat}_3\rangle$, its decomposition of 2 terms will give an equivalent number of terms as the cut decomposition. Thus, the heuristic will always pick the correct cuts, or use the $|\text{cat}_3\rangle$ decomposition. □

While this is not as good as the $O(\frac{\log^2 n}{n}2^n)$ time of the dense IQP circuits, it gives a much closer upper bound than the general one, and is relatively easy to implement in practice. In fact, we can generalise this even further.

Proposition 4.1.8. Suppose we have ZX-diagram in reduced gadget form, which has n T-like spiders not part of phase gadgets, and $\tilde{O}(n^k)$ [1] phase gadgets, for $k \in \mathbb{R}_+$. Then the stabiliser decomposition can be completed in $O(2^n)$ time.

[1] $\tilde{O}(f(n)) = O(f(n)\log^p f(n))$ for some p is a convenient shorthand ignoring polylogarithmic factors

Proof. Simply cut the n T-like spiders to get $O(2^n)$ terms. Since the phase gadgets will now be legless, the diagram is easy to simulate. □

We have that for dense IQP circuits, $k = 2$. The resulting efficiency of just using cuts is then

$$\alpha = \frac{n}{n + \tilde{O}(n^k)} = \frac{1}{1 + \tilde{O}(n^{k-1})} \tag{4.9}$$

It can be noted that for $k \leq 1$, the cut approach will not really be useful, since $n^{k-1} \leq 1$ so its $\alpha \geq 0.5$ as $n \to \infty$. However, for $k > 1$, we see that $\alpha < 0.5$ as $n \to \infty$ is inversely correlated with $n^{k-1} > 1$, and so it might be useful to use a different metric that is not based on the T-count. Instead, as observed in [6], we see that the growth of the number of terms is more like $O(2^{\beta n})$ for some $0 < \beta \leq 1$. For dense IQP circuits, they obtained $\beta \approx 0.93$. For sparse IQP circuits where $k = 1$ (so there were $O(n \log n)$ phase gadgets), they obtained $0.5 < \beta < 0.93$. Compare this to using α where we have the growth being

$$O(2^{\alpha \cdot \tilde{O}(n^k)}) = O\left(2^{\frac{\tilde{O}(n^k)}{\tilde{O}(n^{k-1})}}\right) = O(2^{O(n)}) \tag{4.10}$$

which ends up being equivalent.

Definition 4.1.9. Suppose we have ZX-diagram in reduced gadget form, which has n T-like spiders not part of phase gadgets, and $\tilde{O}(n^k)$ phase gadgets, for $k \in \mathbb{R}_{>1}$. Let p be the number of terms in the stabiliser decomposition D. We define the *cut efficiency* β as

$$\beta(D) = \frac{p}{n} \tag{4.11}$$

▲

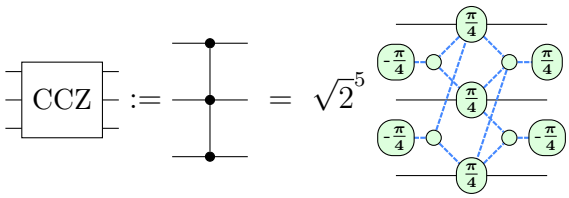
We note that the cut heuristic as defined in (3.22) works very well due to the presence of many $|\text{cat}_3\rangle$. In the general case where this may not be true, the conceptualisation of different heuristics could be developed to exploit any classes of circuits that have a structure such that its reduced gadget form allows for Proposition 4.1.8 to be applied.

4.1.3 Modified Hidden Shift

Hidden shift circuits have been used in various benchmarks since being introduced in [15], starting from the benchmark of the BSS decomposition, and continuing to the benchmark of the $|\text{cat}_n\rangle$ decomposition [2, 3, 18, 27]. However, it was noted that improvements QuiZX trivialised the decomposition of these circuits, with a conjecture that the class of circuits being simulable in polynomial time [4]. Thus, in [28], they introduced a modification to the hidden shift circuits, which they called the *modified hidden shift* circuits.

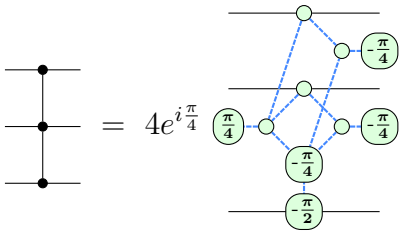
First, we note that the T-count of the hidden shift circuits only comes from the T-spiders appearing CCZ gates in the circuits. We introduce the definition of the CCZ gate, shown as presented in [2].

Definition 4.1.10 (CCZ gate [2])**.** The CCZ gate is the controlled-CZ-gate, defined as



(4.12)

It arises from simplifying the 'textbook' presentation of CCZ or Toffoli in terms of CNOT and T gates (see e.g. [10], Section 4.3). The 4-spider version is the following:

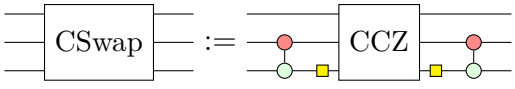


(4.13)

▲

While the structure of the original hidden shift circuits made use of CCZ gates, the modified hidden shift circuits used controlled swap gates instead. The simple addition of the CNOT and Hadamard gates to form the new controlled swap gate made the circuits non-trivial again.

Definition 4.1.11 (Controlled swap gate)**.** The controlled swap (Friedkin) gate is defined as



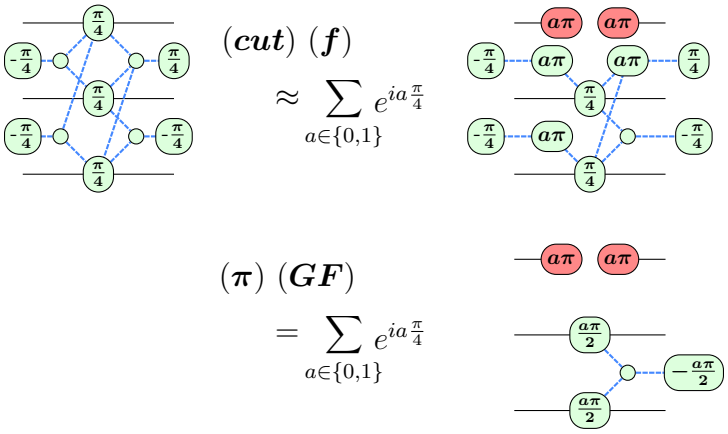
(4.14)

▲

We also have the following useful observation about the CCZ gate.

Lemma 4.1.12. The CCZ gate can be decomposed into 2 stabiliser terms.

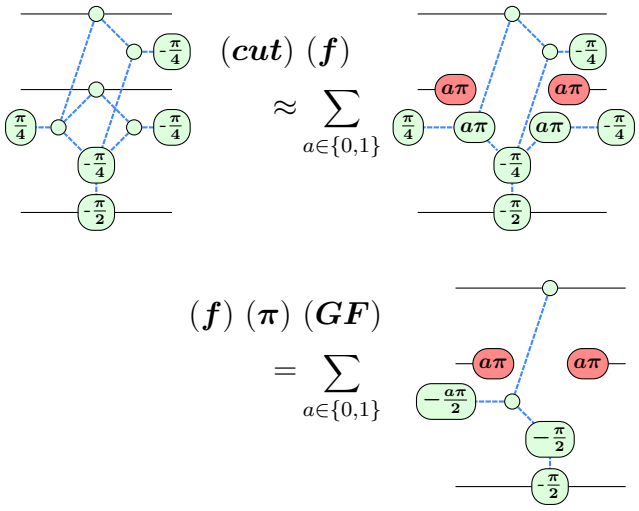
Proof.



(4.15)

□

While at first glance, this seems to give a decent $\alpha = 1/7 \approx 0.143$, we notice from Eq. 4.13 that CCZ gates can be expressed with just 4 T-spiders. In fact, the 2 terms can be derived from the 4 T-spider version.



(4.16)

It is then fairer to say that this cut decomposition has $\alpha = 0.25$.

The structure of CCZ can be visually inspected to see that 2 and 3-legged phase gadgets appear in the circuits. The effectiveness of the heuristic in exploiting this structure can be seen later in this chapter.

4.1.4 Random Clifford+T with CCZ

Last but not least, we consider the combination of the random Clifford+T circuits with CCZ gates, as was benchmarked in [28]. We use the similar distribution of gate sets with 5% each of CCZ and T gates, with the rest of distribution uniform over the Clifford gates (Hadamard, CNOT, CZ, S).

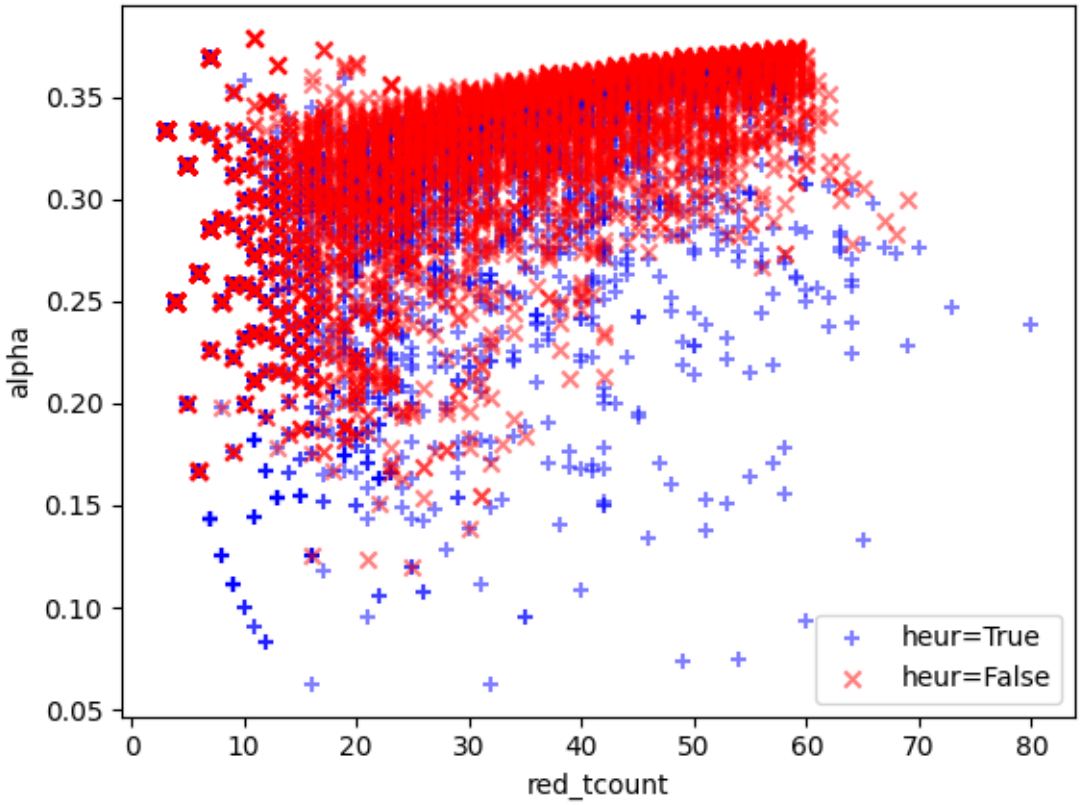


Figure 4.2: α values for all random Clifford+T circuits

4.2 Experiments

We now present the results of the benchmarking of our heuristics. For all experiments, we have a timeout of 90 seconds per circuit simulated.

4.2.1 Random Clifford+T

For these circuits, we can choose the number of qubits q, and the depth d. We stick with the standard minimum and maximum weights of 2 and 4 as in [3]. We take $q = 7, 19, 20, 47, 50, 97, 100$ and $10 \leq d \leq 103$, where we increment d by 3 each time, and take the results of 50 random circuits for each (q, d) pair.

4.2.2 Random IQP

For these circuits, we can choose the number of qubits q. We take $5 \leq q \leq 25$. As mentioned in the previous section, the β effiency is a more useful metric for these circuits.

We also compare this to only using the single cut heuristic.

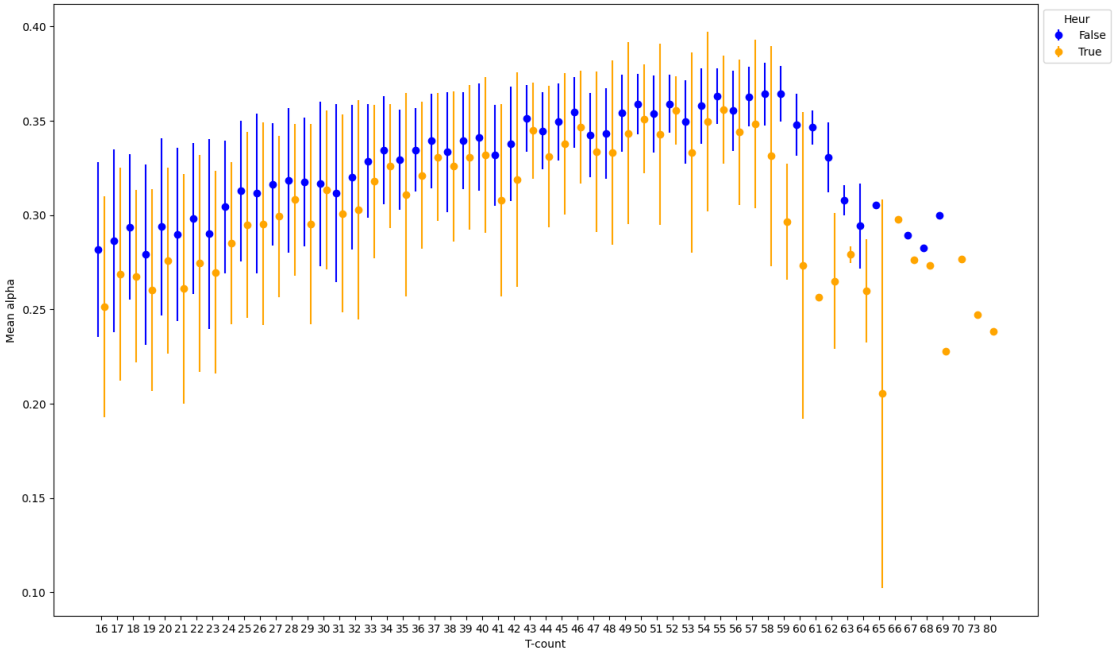


Figure 4.3: Mean α values for different T-counts

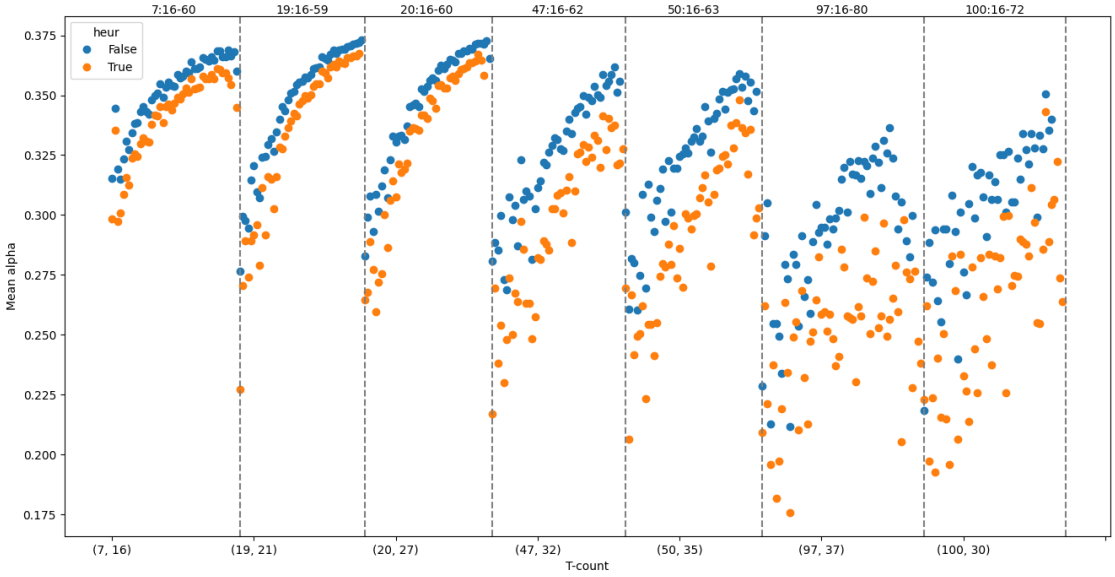


Figure 4.4: Mean α values separated by number of qubits

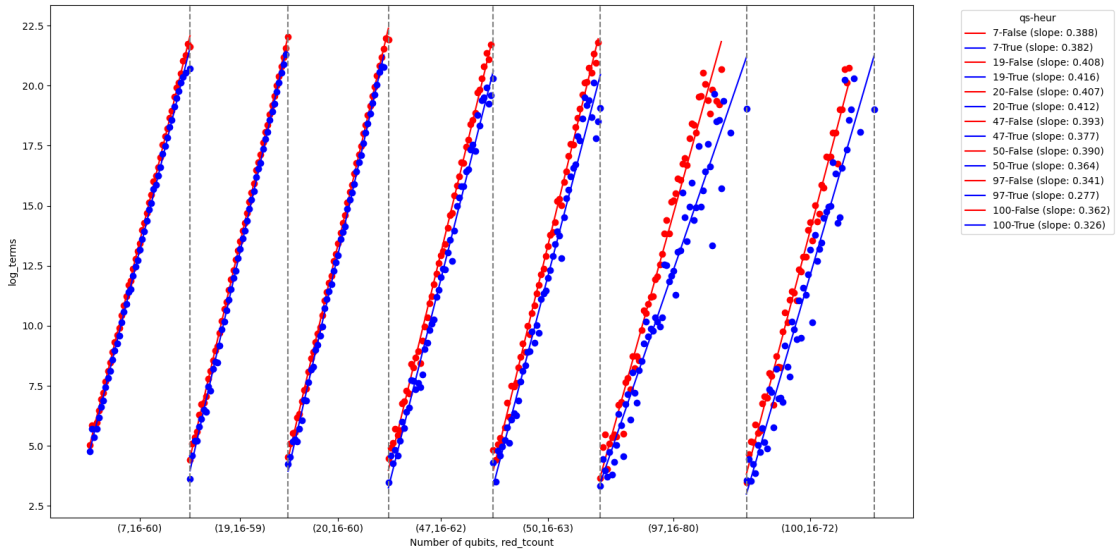


Figure 4.5: Mean log number of terms for different T-counts, separated by number of qubits

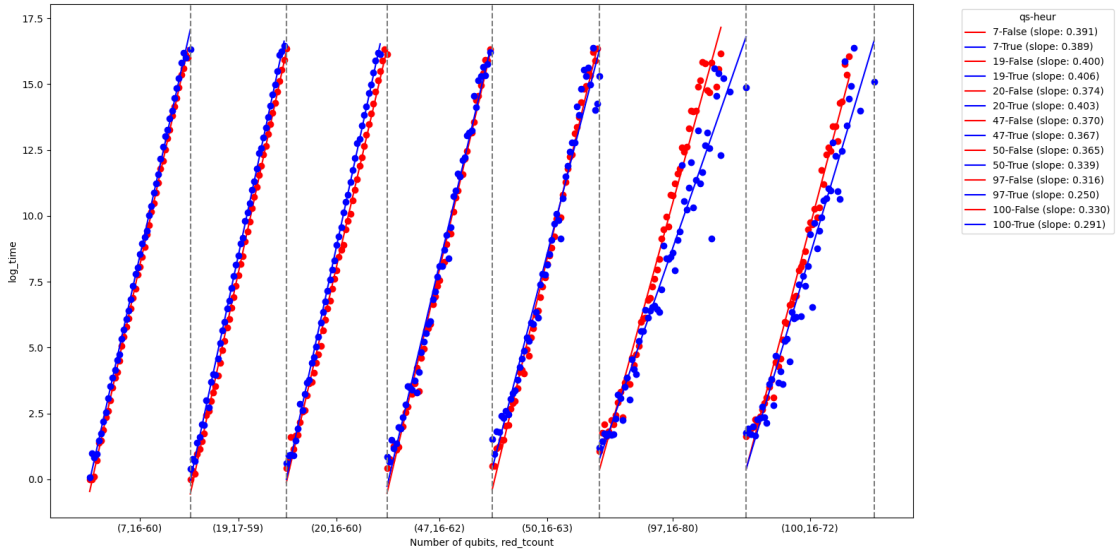


Figure 4.6: Mean log time for different T-counts, separated by number of qubits

4.2.3 Modified Hidden Shift

For these circuits, we can choose the number of qubits q, and the number of CCZ gates n. We take $q = 6, 20, 50$ and $2 \leq n \leq 40$. For $q = 6$, we take $2 \leq n \leq 200$.

4.2.4 Random Clifford+T with CCZ

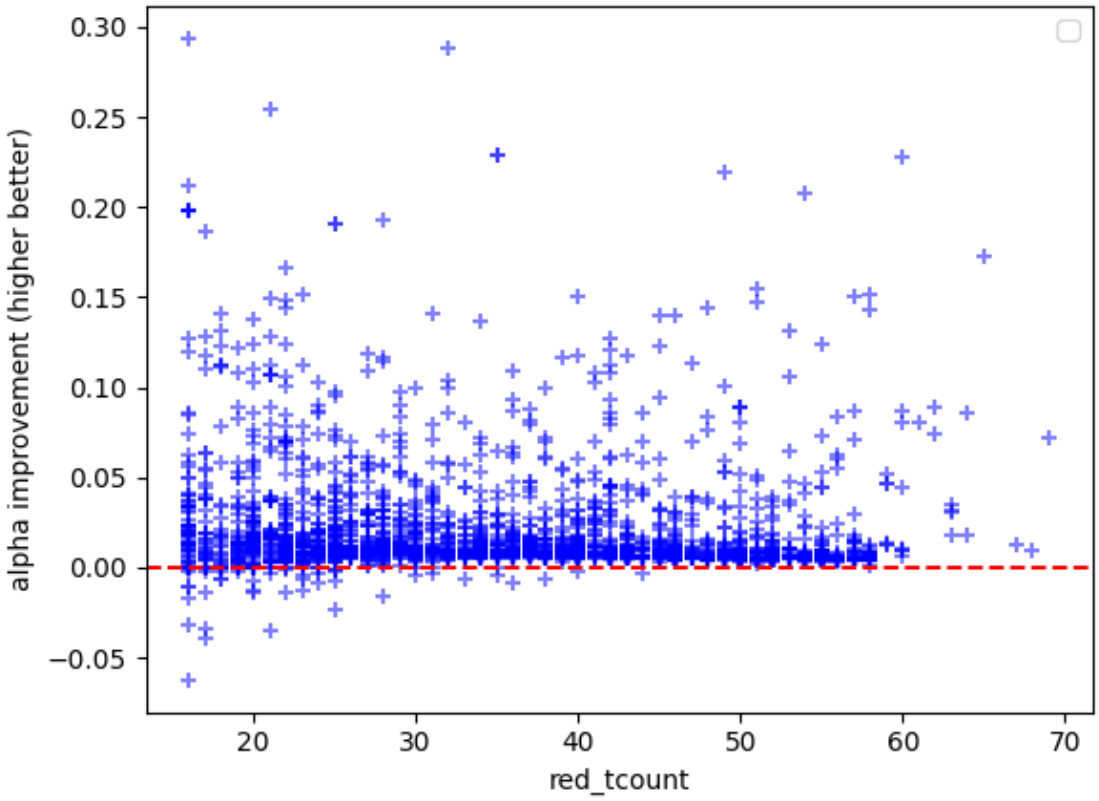


Figure 4.7: Improvement in α values for all random Clifford+T circuits

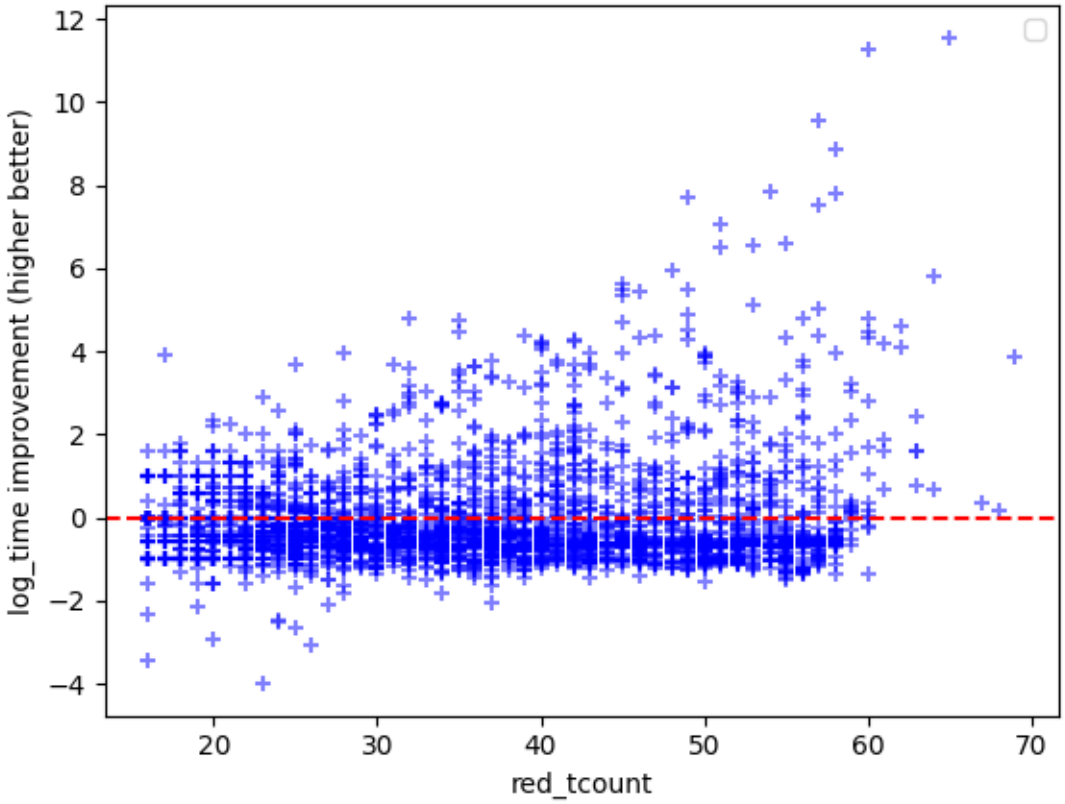


Figure 4.8: Improvement in log time for all random Clifford+T circuits

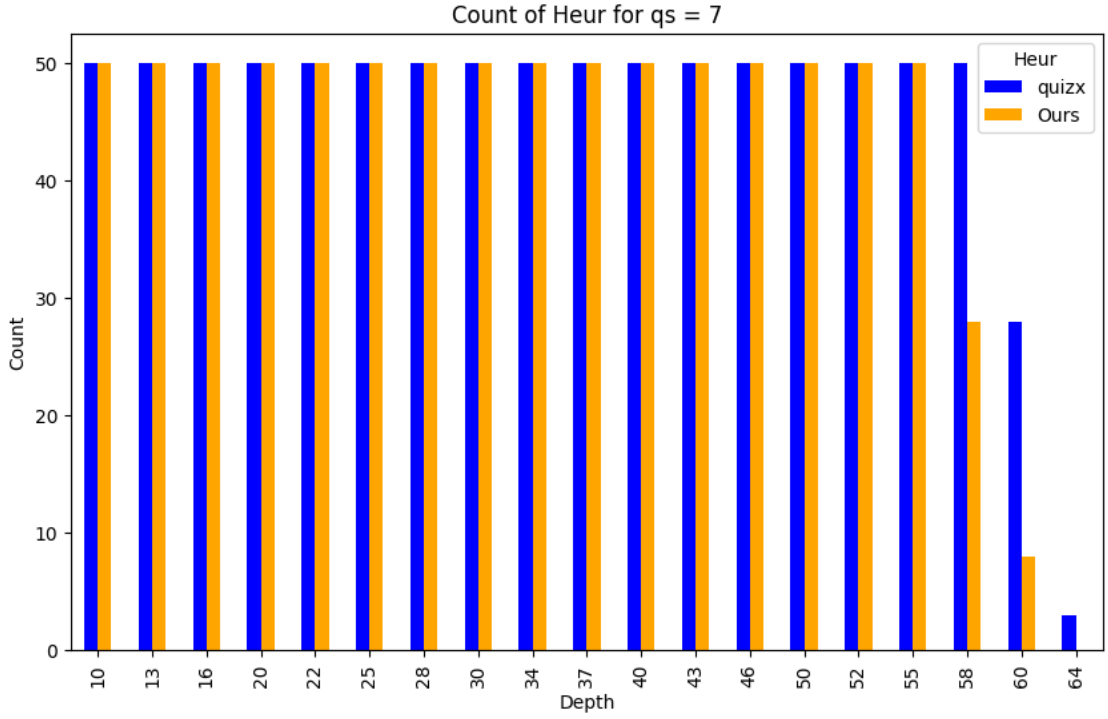


Figure 4.9: Number of completed simulations before timeout for $q = 7$

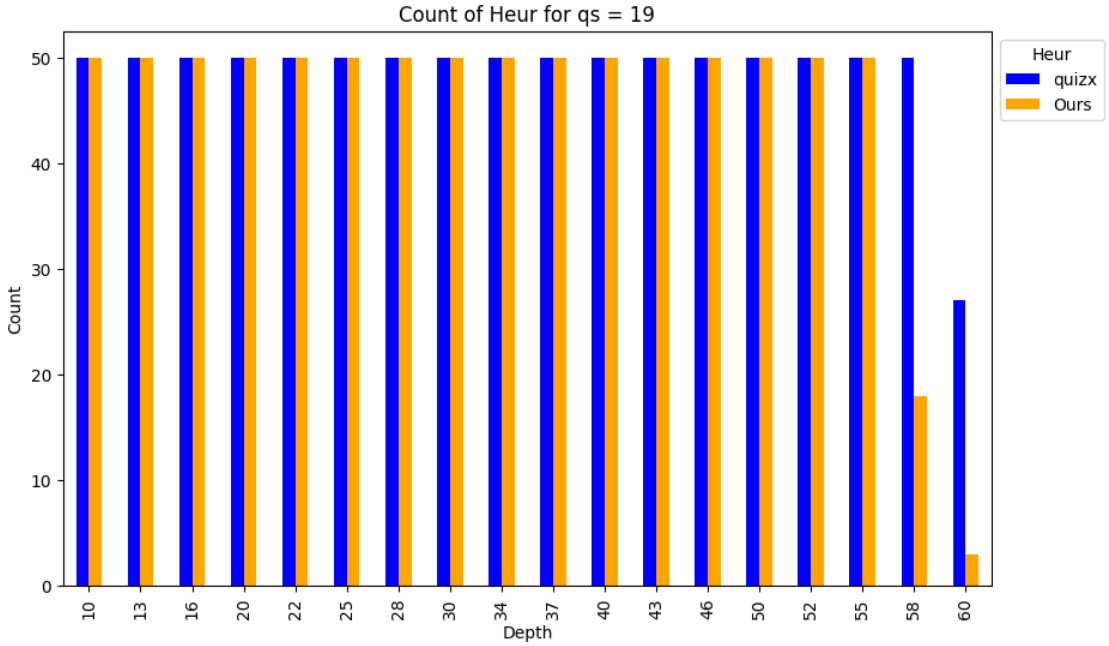


Figure 4.10: Number of completed simulations before timeout for $q = 19$

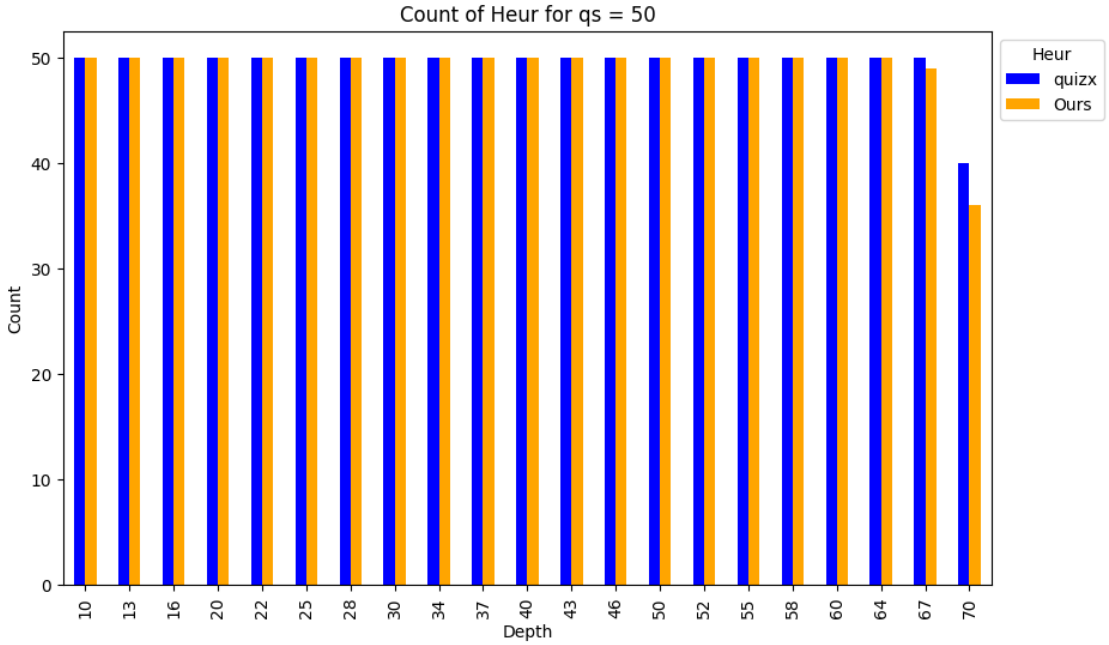


Figure 4.11: Number of completed simulations before timeout for $q = 50$

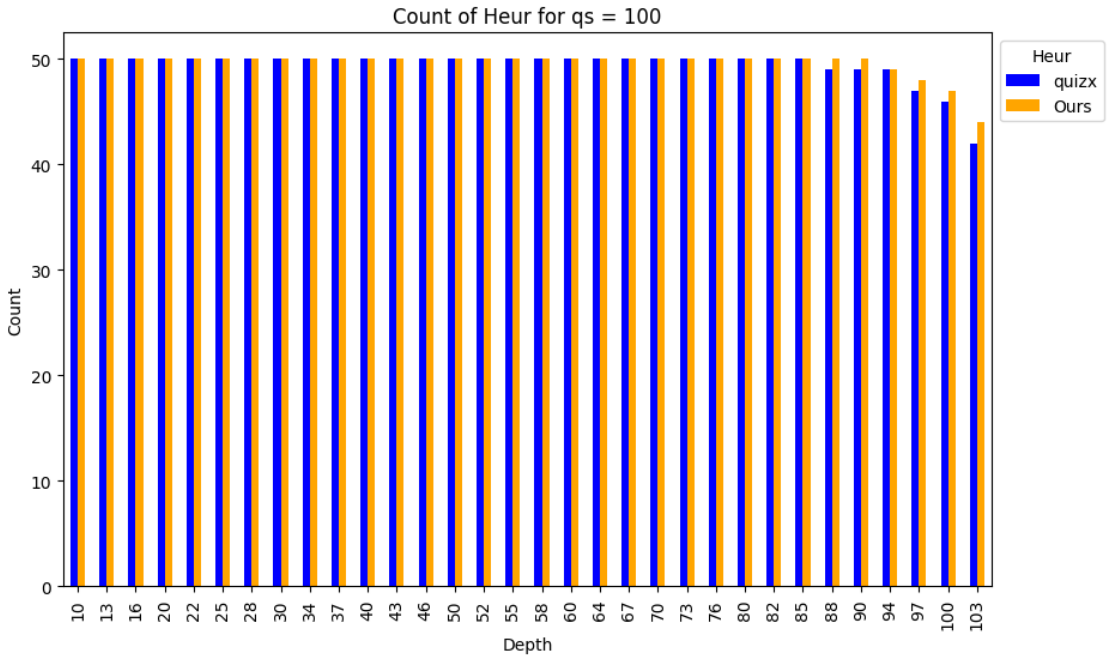


Figure 4.12: Number of completed simulations before timeout for $q = 100$

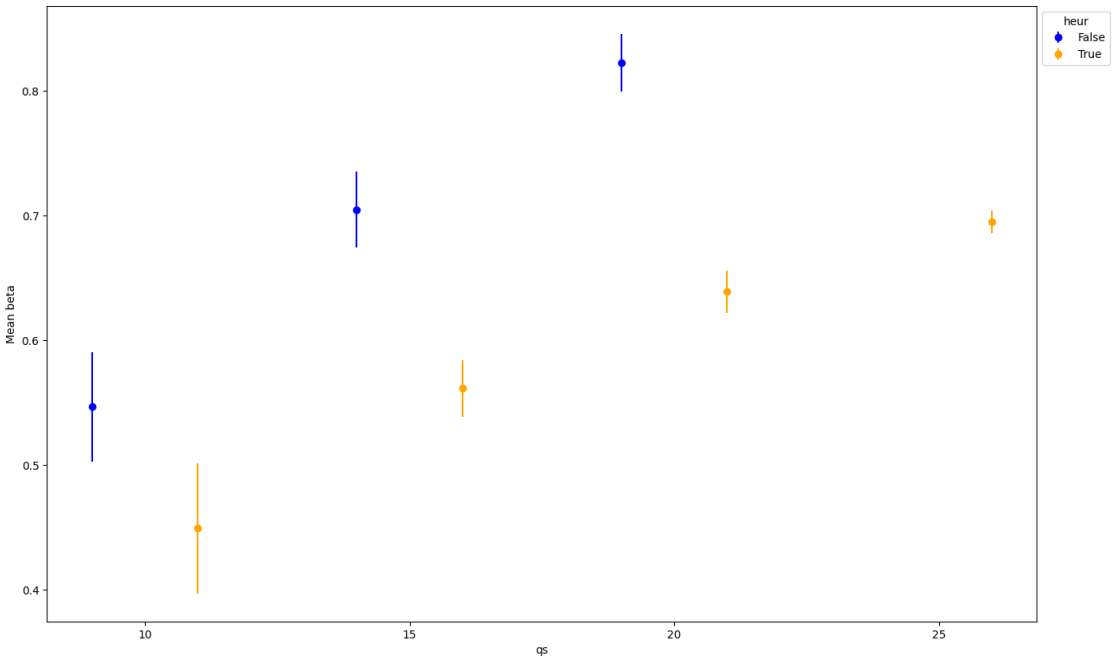


Figure 4.13: Mean β values separated by number of qubits

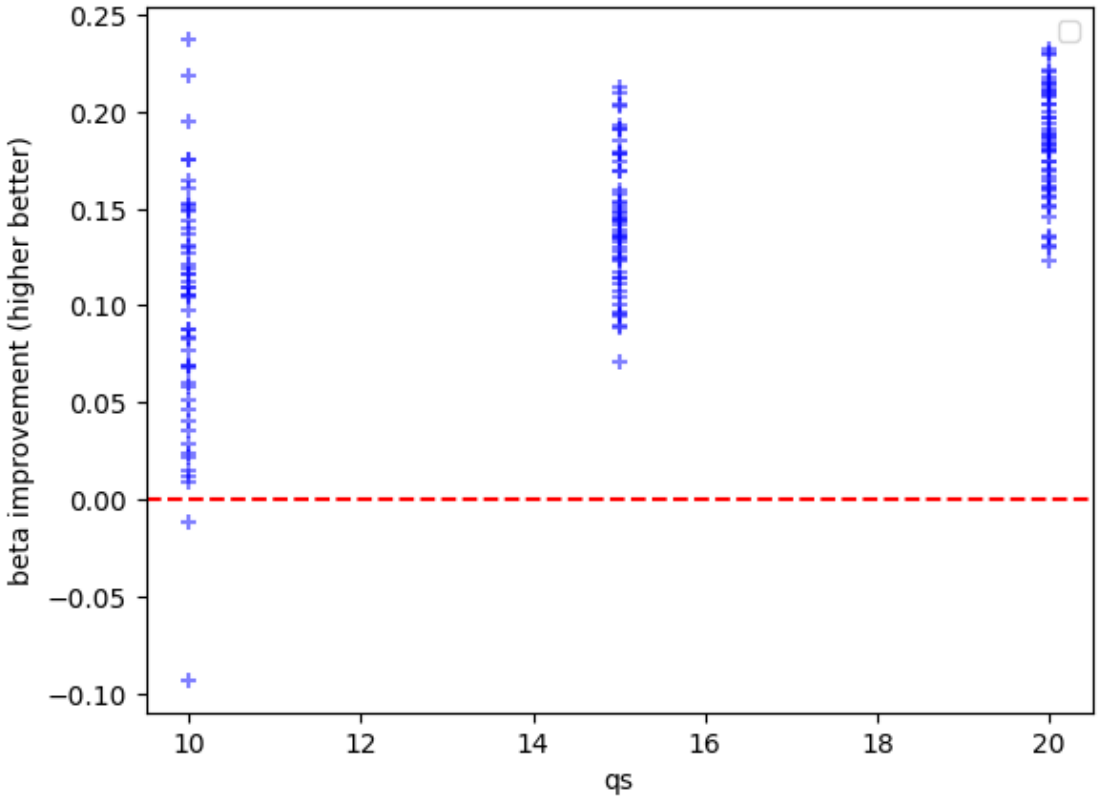


Figure 4.14: Improvement in β values for all random IQP circuits

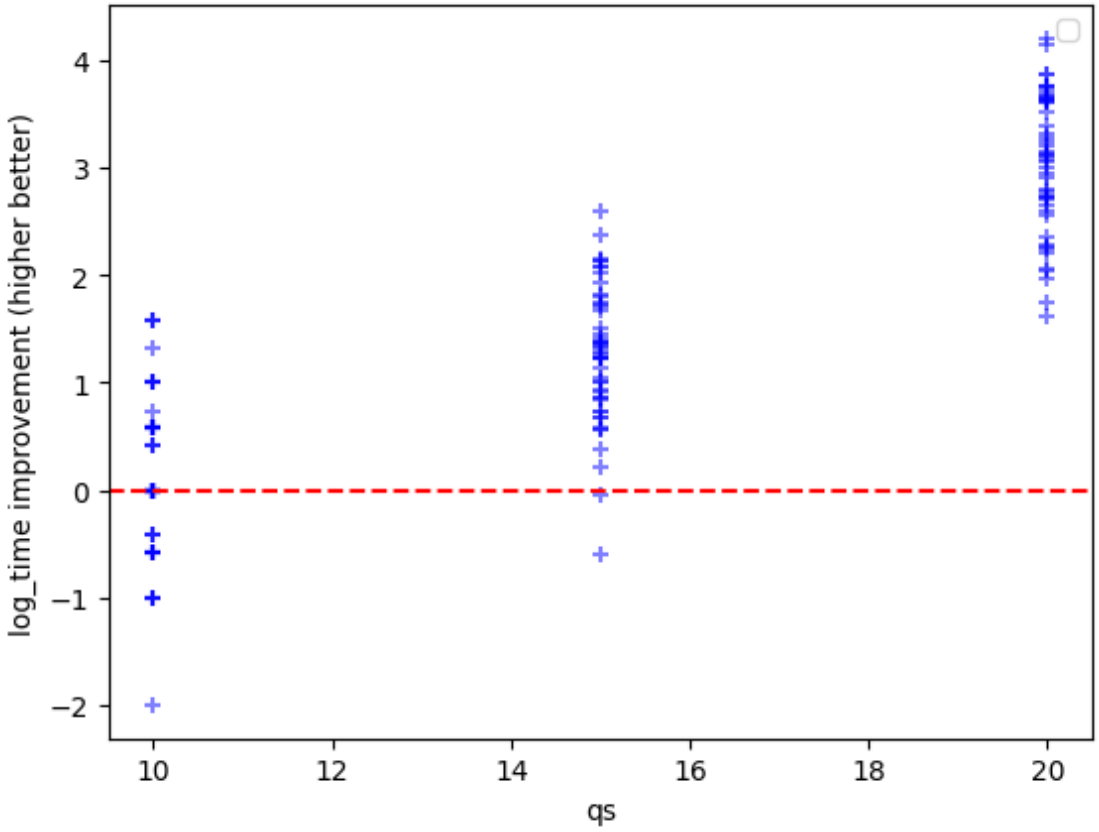


Figure 4.15: Improvement in log time for all random IQP circuits

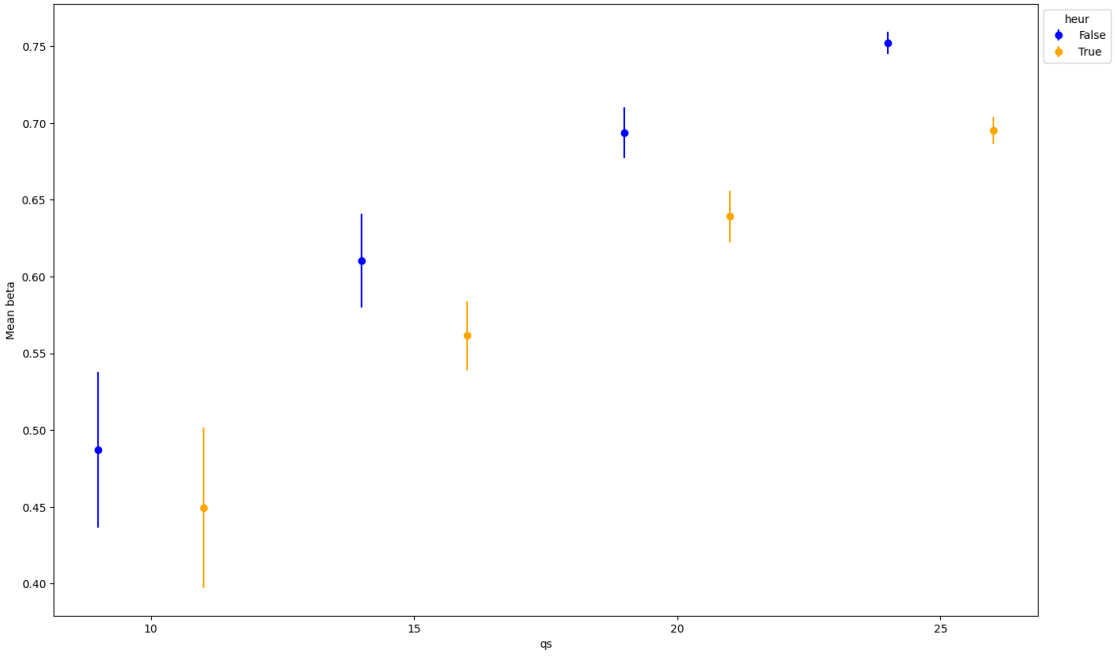


Figure 4.16: Mean β values separated by number of qubits, cut vs combined

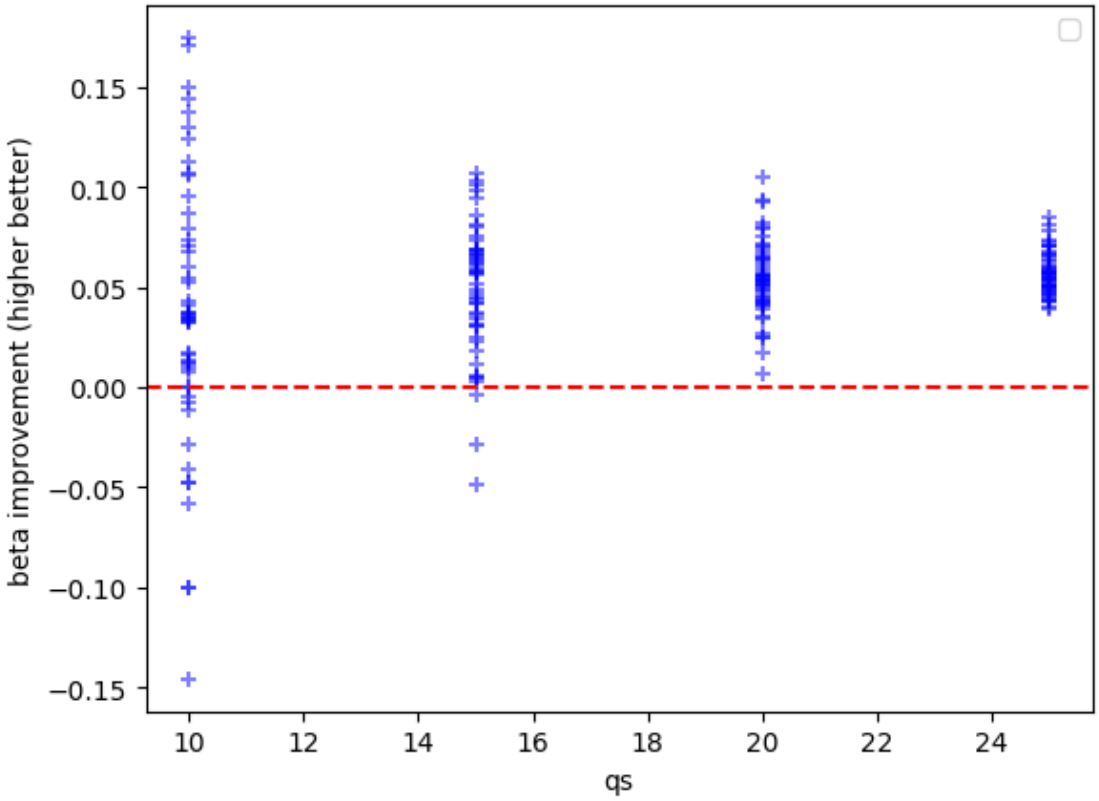


Figure 4.17: Improvement in β values for all random IQP circuits, cut vs combined

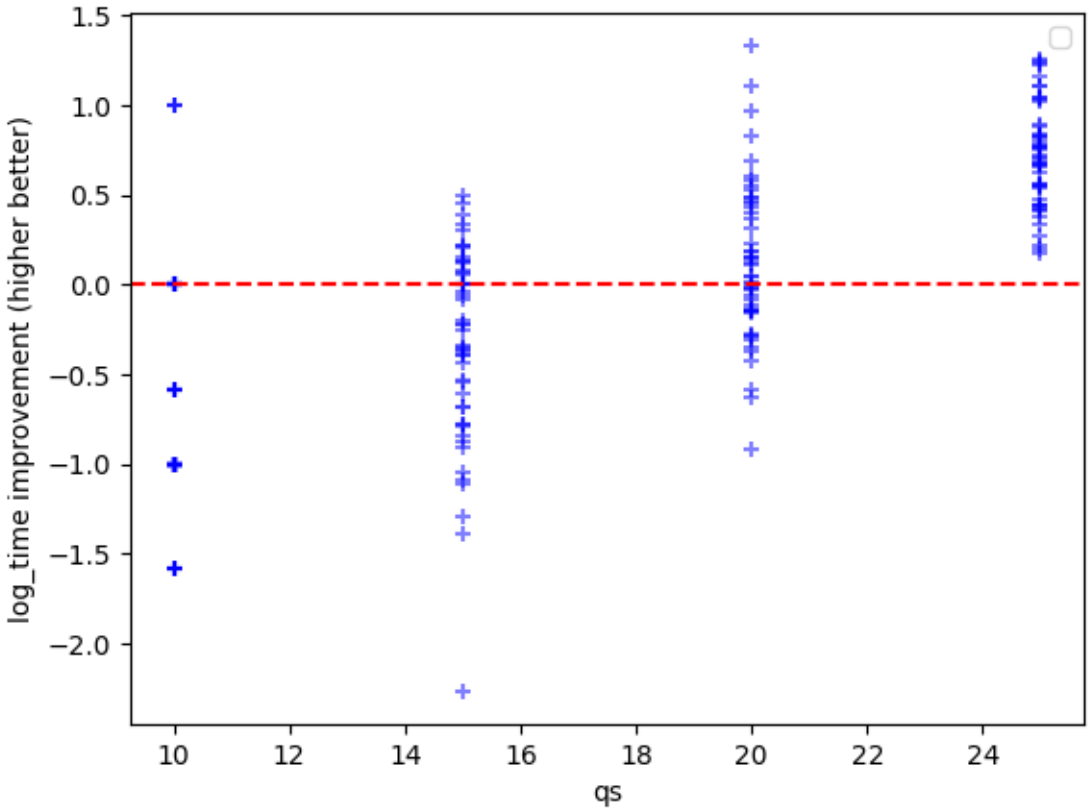


Figure 4.18: Improvement in log time for all random IQP circuits, cut vs combined

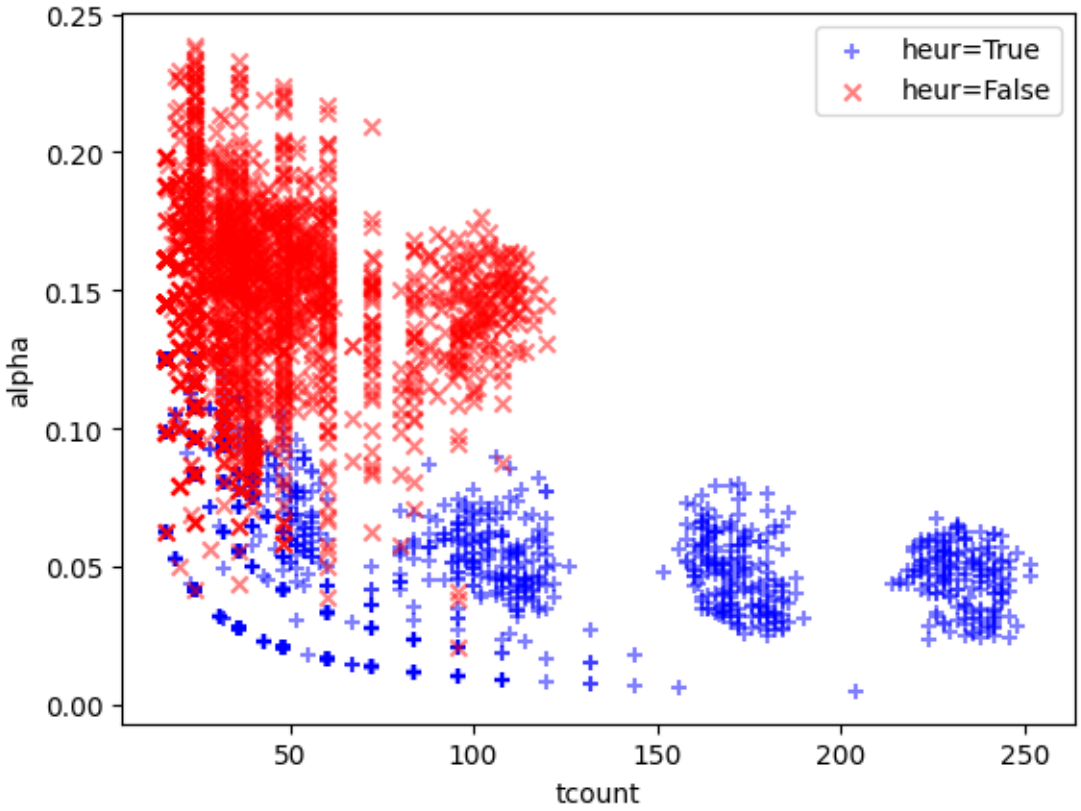


Figure 4.19: α values for all modified hidden shift circuits

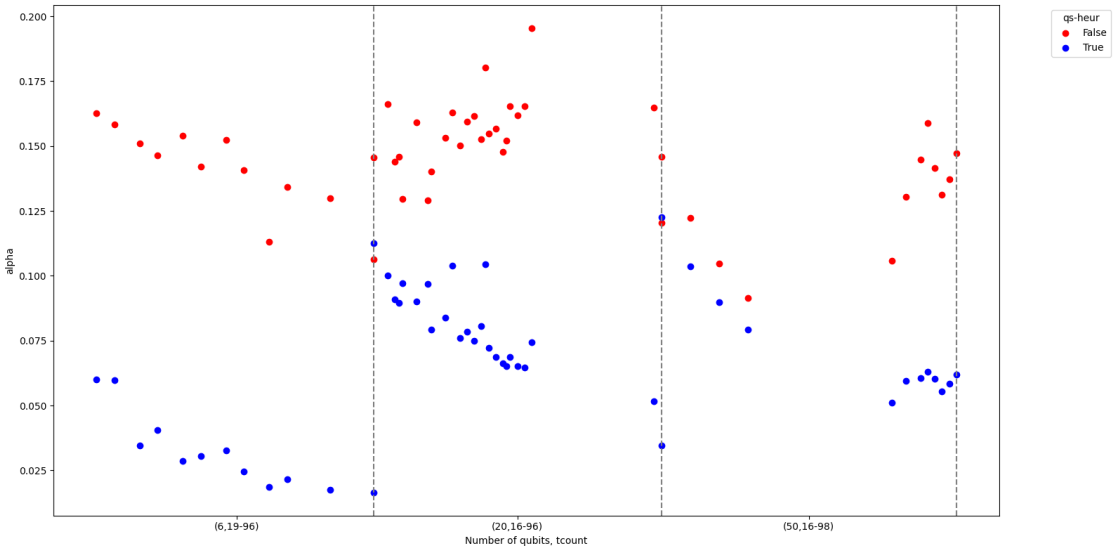


Figure 4.20: Mean α values of modified hidden shift circuits for different T-counts

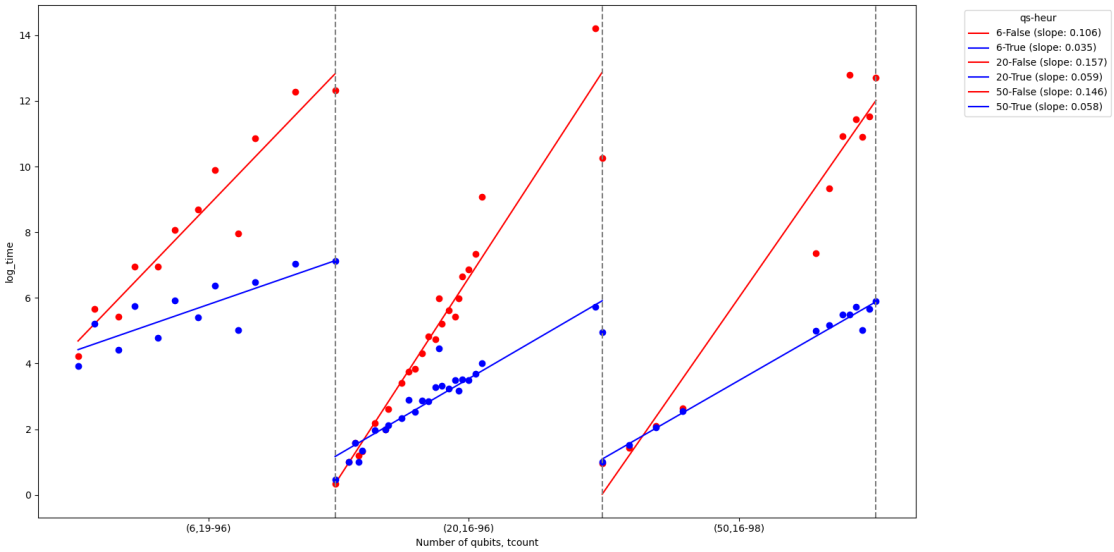


Figure 4.21: Mean log time of modified hidden shift circuits for different T-counts

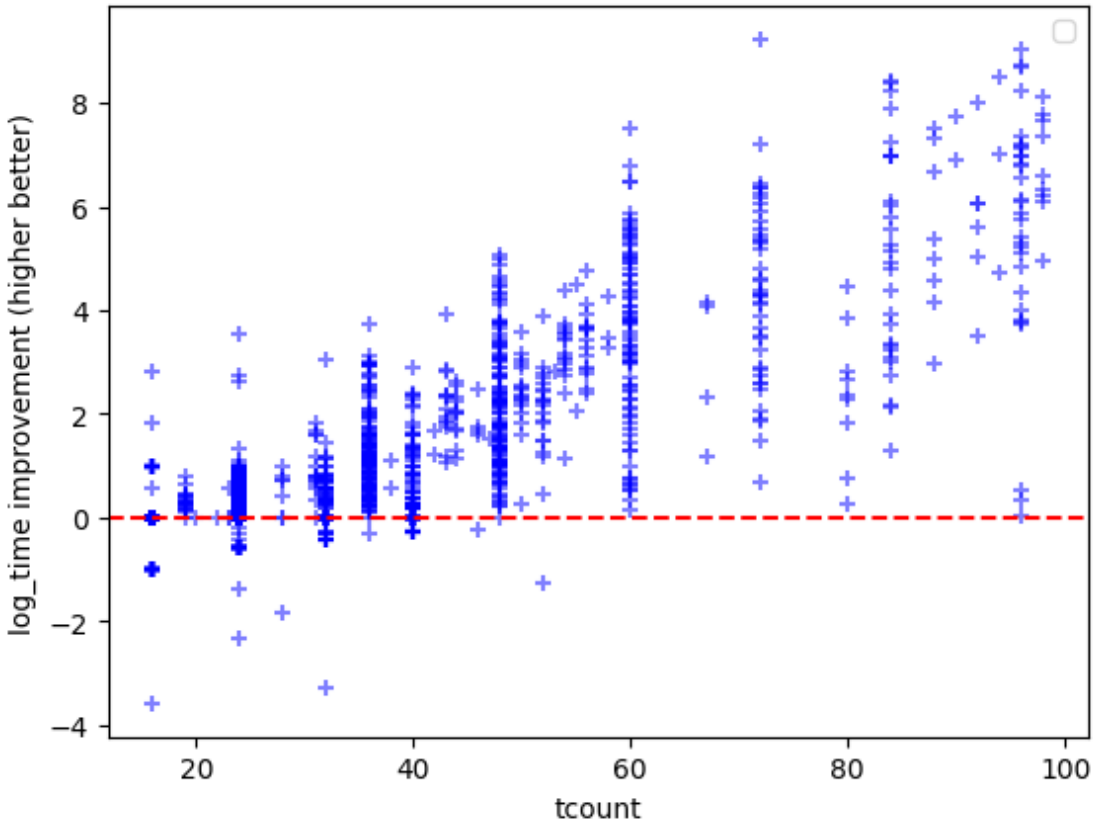


Figure 4.22: Improvement in log time for all modified hidden shift circuits

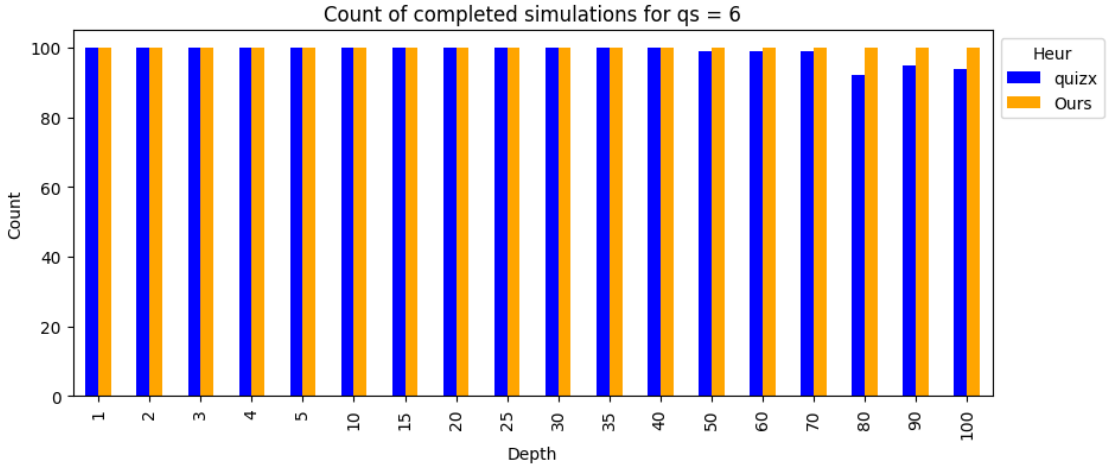


Figure 4.23: Number of completed simulations before timeout for $q = 7$

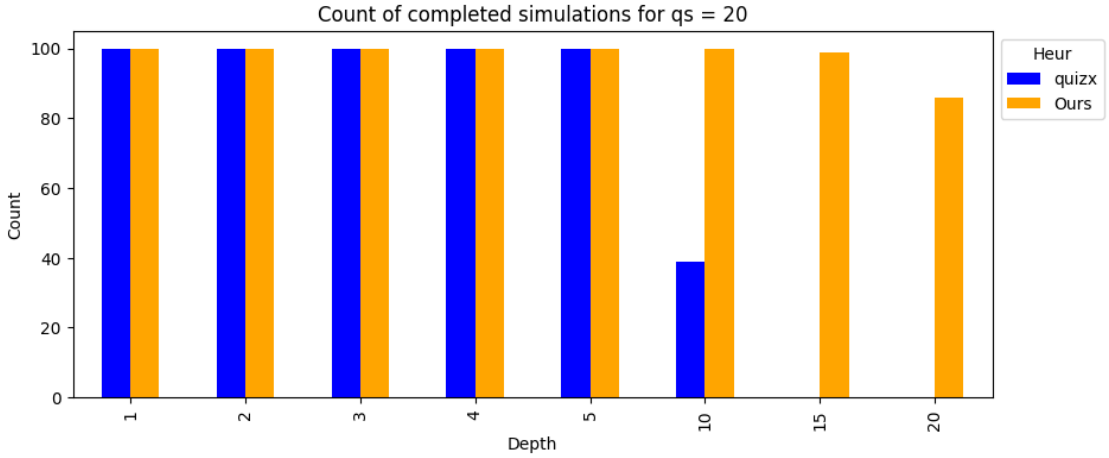


Figure 4.24: Number of completed simulations before timeout for $q = 20$

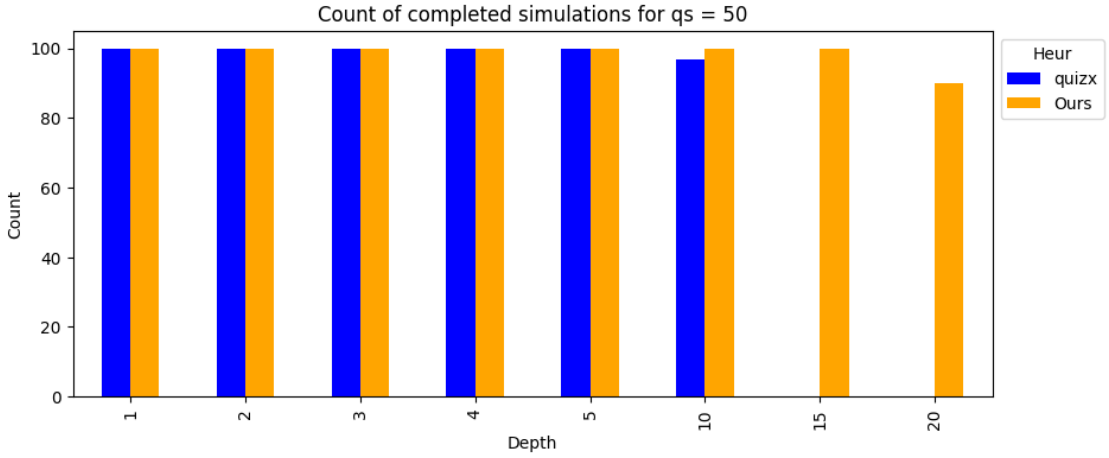


Figure 4.25: Number of completed simulations before timeout for $q = 50$

Appendices

References

[1] Xiaosi Xu et al. *A Herculean task: Classical simulation of quantum computers*. 2023. arXiv: 2302.08880.

[2] Aleks Kissinger and John van de Wetering. "Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions". In: *Quantum Science and Technology* 7.4 (July 2022), p. 044001. URL: http://dx.doi.org/10.1088/2058-9565/ac5d20.

[3] Aleks Kissinger, John van de Wetering, and Renaud Vilmart. "Classical Simulation of Quantum Circuits with Partial and Graphical Stabiliser Decompositions". en. In: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TQC.2022.5.

[4] Julien Codsi. "Cutting-Edge Graphical Stabiliser Decompositions for Classical Simulation of Quantum Circuits". MA thesis. University of Oxford, 2022.

[5] Matthew Sutcliffe and Aleks Kissinger. "Procedurally Optimised ZX-Diagram Cutting for Efficient T-Decomposition in Classical Simulation". In: *Quantum Physics and Logic 2024*. Ed. by Alejandro Díaz-Caro and Vladimir Zamdzhiev. 2024. URL: https://arxiv.org/abs/2403.10964.

[6] Julien Codsi and John van de Wetering. *Classically Simulating Quantum Supremacy IQP Circuits through a Random Graph Approach*. 2023. arXiv: 2212.08609 [quant-ph]. URL: https://arxiv.org/abs/2212.08609.

[7] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes*. Cambridge University Press, 2017. URL: https://books.google.com.sa/books?id=I9gcDgAAQBAJ.

[8] Bob Coecke and Ross Duncan. "Interacting quantum observables: categorical algebra and diagrammatics". In: *New Journal of Physics* 13.4 (Apr. 2011), p. 043016. URL: http://dx.doi.org/10.1088/1367-2630/13/4/043016.

[9] Aleks Kissinger and John van de Wetering. "Universal MBQC with generalised parity-phase interactions and Pauli measurements". In: *Quantum* 3 (Apr. 2019), p. 134. URL: http://dx.doi.org/10.22331/q-2019-04-26-134.

[10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[11] Aleks Kissinger and John van de Wetering. *Picturing Quantum Software*. 2024.

[12] Ross Duncan et al. "Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus". In: *Quantum* 4 (June 2020), p. 279. URL: https://doi.org/10.22331/q-2020-06-04-279.

[13] Aleks Kissinger and John van de Wetering. "Reducing the number of non-Clifford gates in quantum circuits". In: *Phys. Rev. A* 102 (2 Aug. 2020), p. 022406. URL: https://link.aps.org/doi/10.1103/PhysRevA.102.022406.

[14] Hammam Qassim, Hakop Pashayan, and David Gosset. "Improved upper bounds on the stabilizer rank of magic states". In: *Quantum* 5 (Dec. 2021), p. 606. URL: http://dx.doi.org/10.22331/q-2021-12-20-606.

[15] Sergey Bravyi and David Gosset. "Improved Classical Simulation of Quantum Circuits Dominated by Clifford Gates". In: *Physical Review Letters* 116.25 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevLett.116.250501.

[16] Dave Wecker and Krysta M. Svore. *LIQUi|>: A Software Design Architecture and Domain-Specific Language for Quantum Computing*. 2014. arXiv: 1402.4467 [quant-ph]. URL: https://arxiv.org/abs/1402.4467.

[17] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9807006.

[18] Sergey Bravyi et al. "Simulation of quantum circuits by low-rank stabilizer decompositions". In: *Quantum* 3 (Sept. 2019), p. 181. URL: http://dx.doi.org/10.22331/q-2019-09-02-181.

[19] Sergey Bravyi, Graeme Smith, and John A. Smolin. "Trading Classical and Quantum Computational Resources". In: *Physical Review X* 6.2 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevX.6.021043.

[20] Tomoyuki Morimae and Suguru Tamaki. "Fine-grained quantum computational supremacy". In: *Quantum Information and Computation* 19.13-14 (2019), pp. 1089–1115.

[21] John E. Hopcroft and Richard M. Karp. "An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs". In: *SIAM Journal on Computing* 2.4 (1973), pp. 225–231.

[22] Aleks Kissinger, John van de Wetering, and Renaud Vilmart. *QuiZX: a quick Rust port of PyZX*. https://github.com/zxcalc/quizx. 2021.

[23] Aleks Kissinger and John van de Wetering. "PyZX: Large Scale Automated Diagrammatic Reasoning". In: Proceedings 16th International Conference on *Quantum Physics and Logic,* Chapman University, Orange, CA, USA., 10-14 June 2019. Ed. by Bob Coecke and Matthew Leifer. Vol. 318. Electronic Proceedings in Theoretical Computer Science. Open Publishing Association, 2020, pp. 229–241.

[24] David Gosset et al. "An algorithm for the T-count". In: *Quantum Info. Comput.* 14.15–16 (Nov. 2014), pp. 1261–1276.

[25] Michael J. Bremner, Ashley Montanaro, and Dan J. Shepherd. "Achieving quantum supremacy with sparse and noisy commuting quantum computations". In: *Quantum* 1 (Apr. 2017), p. 8. URL: http://dx.doi.org/10.22331/q-2017-04-25-8.

[26] Michael J. Bremner, Ashley Montanaro, and Dan J. Shepherd. "Average-Case Complexity Versus Approximate Simulation of Commuting Quantum Computations". In: *Physical Review Letters* 117.8 (Aug. 2016). URL: http://dx.doi.org/10.1103/PhysRevLett.117.080501.

[27] Filipa C. R. Peres and Ernesto F. Galvão. "Quantum circuit compilation and hybrid computation using Pauli-based computation". In: *Quantum* 7 (Oct. 2023), p. 1126. URL: http://dx.doi.org/10.22331/q-2023-10-03-1126.

[28] Mark Koch, Richie Yeung, and Quanlong Wang. *Speedy Contraction of ZX Diagrams with Triangles via Stabiliser Decompositions*. 2023. arXiv: 2307.01803 [quant-ph]. URL: https://arxiv.org/abs/2307.01803.